Rapport Final de Projet



CRM Alimenté par IA pour Agences Immobilières

- **Réalisé par :**
  - **MOHITO RIAHANA**
  - **ROHAND DOUAE**
  - **AMEZIANE OUMAIMA**
  - **EL BIYADI AMINE**

- **Encadrant :**
  Pr. HACHCHANE Imane

Filière : Génie Informatique - GI2

Année universitaire : 2025 / 2026

Table des matières

Remerciements..... 5
Chapitre 1 - Introduction générale.....6
1.1 Contexte général du projet.....6
1.2 Problématique : les limites de la gestion manuelle dans les agences immobilières.... 6
1.3 Solution proposée : RAWABET.....7
1.4 Objectifs du projet..... 8
Chapitre 2 - Analyse des besoins.....8
2.1 Identification des acteurs.....8
2.2 Besoins fonctionnels..... 9
● Espace Admin :..... 9
● Espace Agent :.....10
● Portail Client :.....10
● Intégration Système & Workflows :..... 10
2.3 Besoins non fonctionnels.....11
Chapitre 3 - Conception.....11
3.1 Choix architectural : le monolithe modulaire.....11
3.2 Découpage en modules..... 12
3.3 Diagramme de classes.....12
3.4 Modèle de données.....13
3.5 Communication inter-modules.....15
Chapitre 4 - Choix technologiques.....15
4.1 Backend :..... 15
4.2 Frontend :.....16
4.3 Base de données..... 18
4.4 Intelligence artificielle..... 18
4.5 Automatisation.....19
4.6 APIs externes.....20
Chapitre 5 - Implémentation.....21
5.1 Module Auth.....21
5.3 Module Moteur IA.....23
5.4 Module Notification.....24
5.5 Portail Client.....24
5.5.1 Landing Page et Acquisition.....24
5.5.2 Authentification et Gestion de Session.....25
5.5.3 Tableau de Bord (Accueil)..... 25
5.5.4 Gestion des Dossiers..... 25
5.5.5 Gestion Documentaire et Contrats.....26
5.5.6 Gestion des Rendez-vous.....27
5.5.7 Assistant IA - MURSHID..... 27
5.5.8 Espace Profil et Sécurité.....27
5.5.9 Centre de Notifications.....28
5.6 Automatisations n8n.....28

5.6.1 Principe d'intégration....28
5.6.2 Workflows implémentés....28
5.6.3 Endpoints exposés par Spring Boot pour n8n....32
5.6.1 Sécurité des webhooks....33
5.7 Captures d'écran....33
5.7.1 Module Auth....33
5.7.2 Landing page d'agent / Admin....34
5.7.3 Espace Agent....34
5.7.4 Espace Admin....38
5.7.5 Portail Client....40
Chapitre 6 - Sécurité....42
6.1 Authentification....42
6.3 Protection des données et défense contre les attaques....43
6.4 Sécurité des APIs externes....43
Chapitre 7 - Répartition des tâches....44
Phase 1 - Conception et modélisation....44
Phase 2 - CRUD & Logique Métier....44
Phase 3 - IA, Automatisation & Intégrations....45
Conclusion générale....47

Remerciements

Nous tenons à adresser nos sincères remerciements à notre professeure Pr. HACHCHANE Imane, pour la qualité de son enseignement, la clarté de ses cours et les connaissances qu'elle nous a transmises tout au long de ce module. Les bases théoriques et pratiques acquises durant ses séances ont été le fondement sur lequel ce projet a été construit.

Nous remercions également chaque membre de l'équipe pour le sérieux, l'engagement et l'esprit de collaboration dont ils ont fait preuve à chaque étape du développement de cette plateforme.

Chapitre 1 - Introduction générale

1.1 Contexte général du projet

Le secteur immobilier est l'un des domaines économiques les plus dynamiques et les plus complexes à gérer. Une agence immobilière traite quotidiennement un volume important d'informations : profils d'acheteurs et de vendeurs, annonces de propriétés, visites programmées, négociations en cours, contrats en attente de signature. La qualité de la gestion de ces informations conditionne directement la performance commerciale de l'agence.
Dans ce contexte, les outils de Gestion de la Relation Client (CRM) jouent un rôle central. Ils permettent de centraliser les données, de structurer les processus commerciaux et de garantir un suivi rigoureux de chaque dossier client. Cependant, la majorité des solutions CRM disponibles sur le marché sont généralistes : elles ne sont pas conçues pour répondre aux spécificités du métier immobilier, et leur adaptation nécessite un effort de personnalisation conséquent.

Par ailleurs, l'essor de l'intelligence artificielle et l'automatisation des processus métier ouvrent de nouvelles perspectives pour les agences : scoring automatique des prospects, analyse intelligente des documents, génération de rapports, et communication automatisée avec les clients. Ces capacités, jusqu'ici réservées aux grandes structures, deviennent accessibles grâce aux APIs modernes de traitement du langage naturel.

C'est dans ce contexte que s'inscrit le projet **Rawabet** : une plateforme CRM conçue spécifiquement pour les agences immobilières, intégrant nativement des fonctionnalités d'intelligence artificielle et d'automatisation des workflows.

1.2 Problématique : les limites de la gestion manuelle dans les agences immobilières

Les agences immobilières de taille moyenne font face à plusieurs problèmes récurrents dans leur gestion quotidienne :

1.2.1 Fragmentation de l'information

Les données clients sont souvent réparties entre des tableurs Excel, des carnets de notes, des boîtes email et des messageries instantanées. Cette dispersion rend difficile le suivi cohérent d'un dossier dans le temps, surtout lors d'un changement d'agent.

1.2.2 Absence de suivi structuré du pipeline

Sans outil dédié, le directeur d'agence ne dispose pas d'une vision globale et en temps réel de l'état de tous les dossiers en cours. Les leads froids passent inaperçus, les opportunités chaudes ne sont pas priorisées, et les dossiers bloqués ne sont identifiés que tardivement.

1.2.3 Processus manuels chronophages

L'envoi d'emails de bienvenue, les rappels de rendez-vous, les relances après signature de contrat, les alertes de leads inactifs - toutes ces tâches sont exécutées manuellement, ce qui consomme du temps et génère des oublis.

1.2.4 Manque d'intelligence sur les données

Les agents disposent de volumes importants de données sur leurs clients (historique d'interactions, documents financiers, préférences) mais n'ont aucun outil pour en extraire des insights actionnables : quel client prioriser ? Quelle propriété recommander ? Ce client est-il financièrement éligible **?**

1.2.5 Risque lié aux départs d'agents

Lorsqu'un commercial quitte l'agence, ses dossiers clients risquent d'être perdus ou mal transmis si aucune règle de continuité n'est en place.

Ces problèmes cumulés entraînent une perte de leads, une dégradation de l'expérience client, et une sous-performance commerciale évitable.

1.3 Solution proposée : RAWABET

Rawabet est une plateforme CRM sectorielle, alimentée par l'intelligence artificielle, conçue exclusivement pour les agences immobilières. Elle répond directement aux problèmes identifiés en combinant trois dimensions complémentaires .

- **Gestion structurée et centralisée**

Chaque dossier client est géré dans un espace unifié : profil complet, historique des interactions, propriétés liées, documents uploadés, contrats et statuts. Toutes les données appartiennent à l'agence, pas à l'agent, ce qui garantit la continuité même en cas de turnover.

- **Couche d'intelligence artificielle intégrée**

Le moteur IA de Rawabet fournit, en temps réel, un score de lead (0 à 100) avec explication en langage naturel, un résumé automatique des interactions de chaque client, des suggestions de mise à jour d'étape pipeline, des recommandations de propriétés personnalisées, une analyse des risques contractuels, et des réponses aux questions en langage naturel sur les documents des clients via un pipeline RAG.

- **Automatisation complète des processus répétitifs**

Via n8n, les workflows critiques sont entièrement automatisés : emails de bienvenue et de suivi, invitations Google Calendar, alertes de leads froids, relances contractuelles, et rapport hebdomadaire livré chaque lundi à l'administrateur.

La conception et les choix technologiques de la plateforme - architecture Spring Boot monolithique modulaire, base de données PostgreSQL avec pgvector, et interfaces React distinctes pour les agents et les clients - font l'objet des chapitres suivants.

1.4 Objectifs du projet

Ce projet poursuit trois objectifs principaux :

1.4.1 Objectif technique :

Mettre en œuvre une architecture logicielle robuste et modulaire, intégrant des technologies modernes : Spring Boot 3, Spring AI, OpenAI GPT-OSS-120B et GPT-OSS-20B , pgvector, n8n, React 18 - et démontrer la maîtrise de l'intégration de systèmes hétérogènes.

1.4.2 Objectif fonctionnel

Concevoir et implémenter une plateforme CRM complète couvrant l'ensemble du cycle de vie d'un dossier client en agence immobilière : création du profil, suivi du pipeline, gestion des rendez-vous et des propriétés, rédaction et suivi des contrats, communication client.

1.4.3 Objectif IA

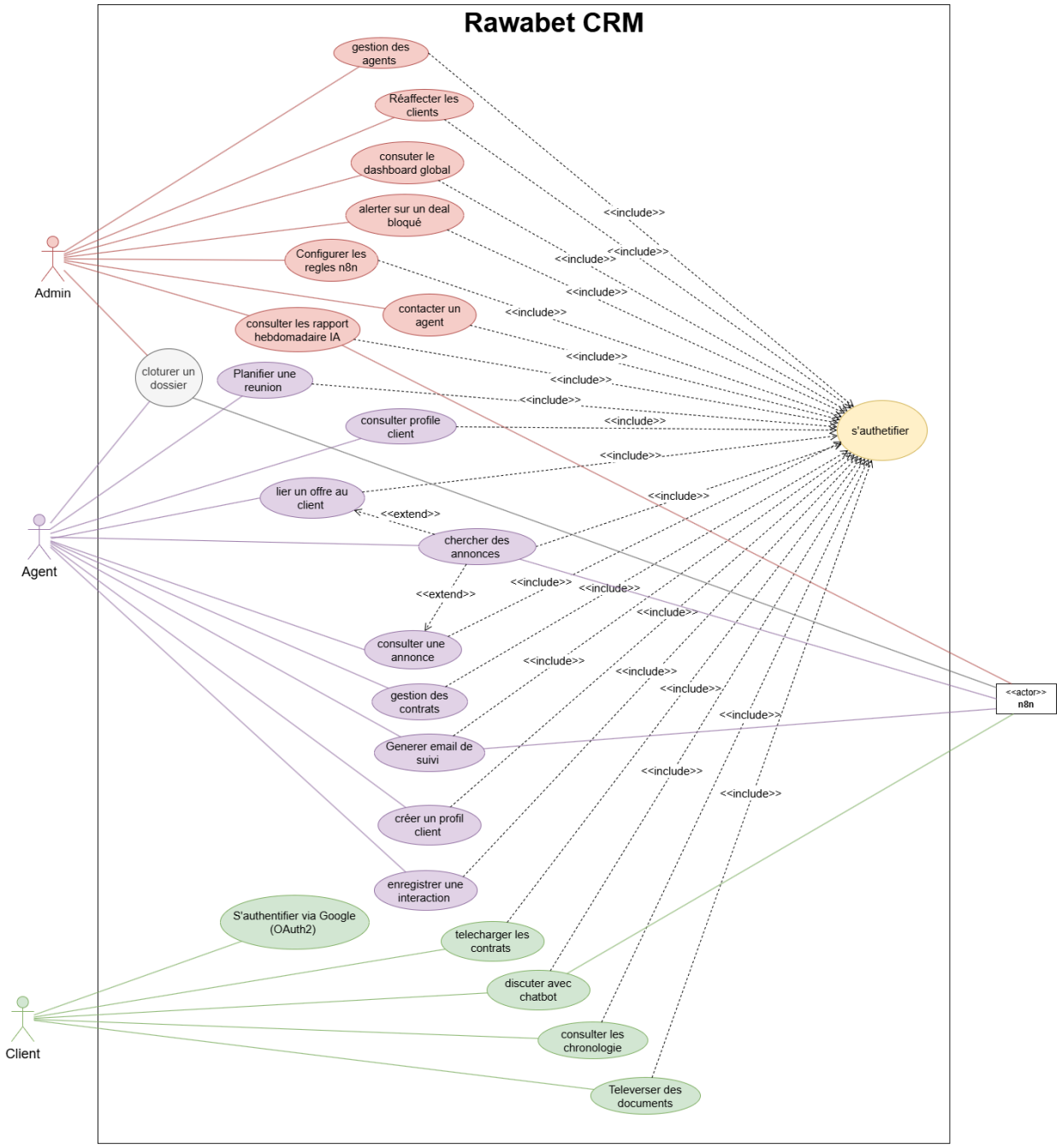
Implémenter un pipeline d'intelligence artificielle opérationnel incluant le scoring de leads, la génération augmentée par récupération (RAG) sur les documents clients, et la génération automatique de rapports - démontrant une application concrète des LLMs dans un contexte métier réel.

Chapitre 2 - Analyse des besoins

2.1 Identification des acteurs

L'application Rawabet repose sur une architecture multi-rôles permettant de segmenter les responsabilités. Nous avons identifié quatre acteurs principaux, dont *trois utilisateurs finaux* et *un acteur système* :

- **Admin :** Il possède les droits d'administration globaux. Son périmètre d'accès inclut la gestion des utilisateurs (agents), la supervision globale du pipeline d'activités, l'affectation ou réaffectation dynamique des clients en cas d'indisponibilité d'un agent, l'alerting sur les deals bloqués et l'accès aux rapports hebdomadaires générés par l'IA.
- **Agent :** Acteur central du CRM, il gère les relations commerciales et le cycle de vie des transactions immobilières. Il est responsable de la création des profils clients, de la planification des rendez-vous, de l'association entre les annonces immobilières et les requêtes des clients, du suivi des interactions, et de la génération des contrats.
- **Client :** L'utilisateur final qui utilise le portail pour consulter l'avancement de son dossier (chronologie), télécharger les contrats, téléverser des documents requis, et discuter en ligne avec le chatbot intelligent du CRM.
- **n8n (Acteur Système) :** Outil central d'automatisation externe. Il orchestre les workflows de notifications, l'extraction des leads inactifs pour relance et agit en tant qu'intermédiaire pour les actions automatisées (génération d'ébauches d'emails via l'IA, alertes admin).



2.2 Besoins fonctionnels

Afin de fluidifier la navigation et d'isoler la logique métier par profil, les exigences fonctionnelles sont catégorisées par "Espace" ou portail d'accès :

- **<u>Espace Admin :</u>**

- **Supervision & Analytique :** Accès à un tableau de bord global (Dashboard) pour suivre la performance de l'agence, et consultation de rapports d'activité analytiques hebdomadaires générés par l'IA.

- **Gestion des Agents & Accès :** Création, suspension et paramétrage des comptes agents, y compris la génération automatique de leurs credentials (envoyés via workflow n8n).

- **Supervision du Pipeline :** Vue globale sur les dossiers du CRM. Capacité d'alerter sur un deal bloqué et de réaffecter des clients à un nouvel agent.

● Espace Agent :

- **Gestion du Portefeuille Client (CRM Core) :** Création de profils clients, définition des critères de recherche et suivi des statuts (acquéreur ou vendeur).
- **Gestion des Interactions & Agenda :** Journalisation complète des appels, réunions et emails. Planification de rendez-vous (Meeting) avec génération de résumés de réunion.
- **Recherche & Liaison Immobilière :** Fonctionnalités de recherche avancée pour associer des offres (Property) et générer des Deal (Transactions) adaptés au besoin du client.
- **Gestion Documentaire & Contractuelle :** Création, génération (PDF) et suivi des contrats (Contract). Suivi du statut de la signature.
- **Assistance IA (Copilot) :** Outils avancés intégrés incluant la génération automatique de brouillons d'emails de suivi via des requêtes spécifiques depuis l'application.

● Portail Client :

- **Authentification Simplifiée (SSO) :** Connexion rapide via l'intégration Google OAuth2 (compte Gmail) avec prise en charge de la liaison de compte depuis le profil.
- **Suivi de Dossier (Chronologie) :** Accès en temps réel à l'historique et au statut des propriétés associées, ainsi qu'aux visites planifiées.
- **Gestion Documentaire & Contractuelle :** Capacité de téléverser des documents justificatifs, de télécharger et de valider/signer les contrats liés à leur dossier (intégration Cloudinary).
- **Assistance Automatisée (Chatbot) :** Interaction textuelle avec un chatbot basé sur l'IA (via l'intégration du Moteur IA) pour obtenir des informations rapides ou planifier des rendez-vous.

● Intégration Système & Workflows :

- **Traitement Asynchrone (CRM → n8n) :** Déclenchement de webhooks sortants lors d'événements clés (Création de client, Réunion planifiée, Contrat signé, Dossier validé).
- **Pilotage entrant (n8n → CRM) :** Capacités pour n8n de requêter le CRM afin d'obtenir des données (ex: rapports hebdomadaires IA, leads inactifs) ou d'agir (clôture de dossier, envoi in-app d'alertes).

2.3 Besoins non fonctionnels

Les exigences de qualité et de contraintes techniques qui garantissent la viabilité de l'application :

- **Sécurité & Confidentialité :** Mise en place d'une authentification hybride basée sur JSON Web Tokens (JWT) et le standard externe OAuth2 (Google Sign-In). Contrôle d'accès basé sur les rôles (RBAC) strict entre les espaces Admin, Agent et Client.
- **Performance & Robustesse :** Utilisation d'un pool de connexions HikariCP optimisé (limité à 5 connexions max par instance) adapté à l'infrastructure cloud pour prévenir les surcharges.
- **Disponibilité & Évolutivité :** Architecture en Monolithe Modulaire garantissant des temps de réponse faibles et permettant une séparation logique prête pour d'éventuelles évolutions vers des microservices.
- **Maintenabilité :** Code organisé par domaine d'affaire. Découplage complet de la logique métier de bas niveau (Spring Events pour la communication inter-domaine).
- **Traçabilité & Journalisation :** Utilisation intensive de Spring Application Events pour créer une "piste d'audit" (Audit Trail) des actions critiques : signatures, modifications de dossiers et échanges de données avec n8n.

Chapitre 3 - Conception

3.1 Choix architectural : le monolithe modulaire

Dans le cadre d'une équipe de développement composée de 4 personnes et compte tenu de nos échéances académiques, le choix d'une architecture orientée microservices aurait impliqué une charge opérationnelle excessive (gestion de multiples dépôts, orchestration réseau complexe, traitement délicat de la consistance distribuée).

C'est pourquoi nous avons opté pour le modèle du Monolithe Modulaire. Les avantages majeurs de cette approche sont multiples :

1. **Cohésion et Agilité :** Tout le code réside dans un même dépôt et s'exécute au sein d'une seule Java Virtual Machine (JVM).
2. **Découplage Logique Fort :** La base de code est strictement cloisonnée par domaine métier. La communication s'effectue via des appels stricts d'interfaces et des événements asynchrones (Spring Events) au lieu de faire des jointures complexes base de données.
3. **Préparation à la Micro-stratification :** En imposant la règle d'or selon laquelle "chaque entité appartient à un et un seul module", et que les références aux autres domaines ne se font que de manière lâche (par l'ID et non par des dépendances d'objets), le basculement d'un module vers un microservice autonome dans le futur

se fera par la simple redéfinition du bus d'événements (par exemple via Kafka ou RabbitMQ à la place de Spring Events), moyennant peu de refactorisation.

3.2 Découpage en modules

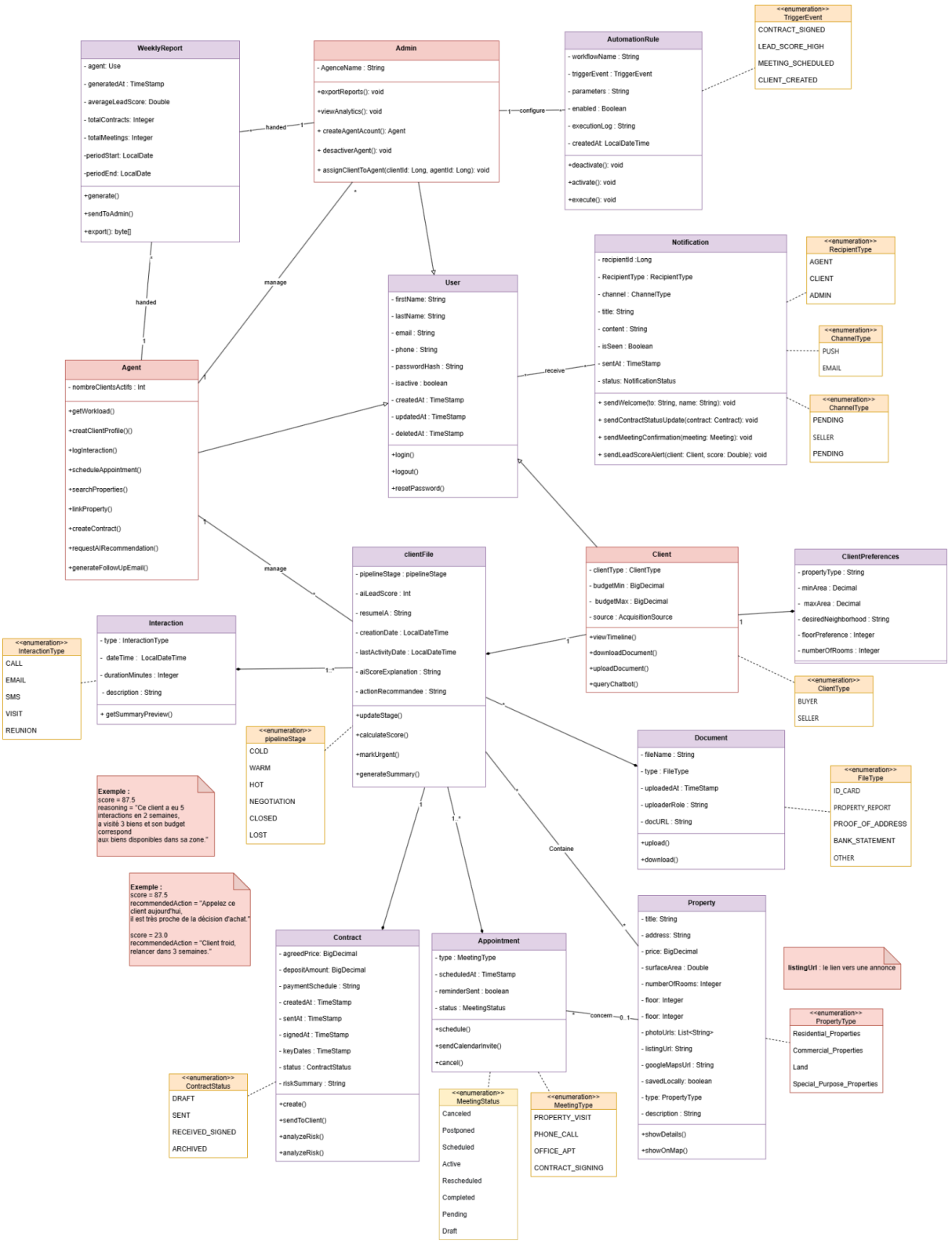
L'architecture logicielle s'articule autour de quatre packages racines principaux :

1. **Auth (com.smartestatehub.auth) :** Centre de sécurité. Gère les *InternalUser(entity/table dans notre base de données regroupant l'admin et les agents avec les rôles)*, les identités, la validation des tokens JWT, la vérification des permissions (rôles), et l'état des sessions.
2. **CRM Core (com.smartestatehub.crm) :** Cœur de l'application. Gère la logique liée à l'immobilier, allant du profil Client, des biens (Property), aux opportunités (Deal, Offer) ou des évènements calendaires (Meeting, Interaction).
3. **Moteur IA (com.smartestatehub.ai) :** Module "Smart" de l'application (LLM via base RAG). Responsable de la génération de réponses intelligentes (Chat), du Smart Lead Scoring, de la rédaction de drafts d'emails et des synthèses de réunions. Utilise intensivement une base vectorielle (pgvector).
4. **Notification / Workflow (com.smartestatehub.notification) :** Interface de commutation avec le monde extérieur. Gère le cycle de Webhooks entrant depuis et sortant vers l'orchestrateur (n8n), déclenchant l'envoi des emails (SendGrid).

3.3 Diagramme de classes

Le diagramme de classes expose les contrats principaux qui composent l'ossature du système :

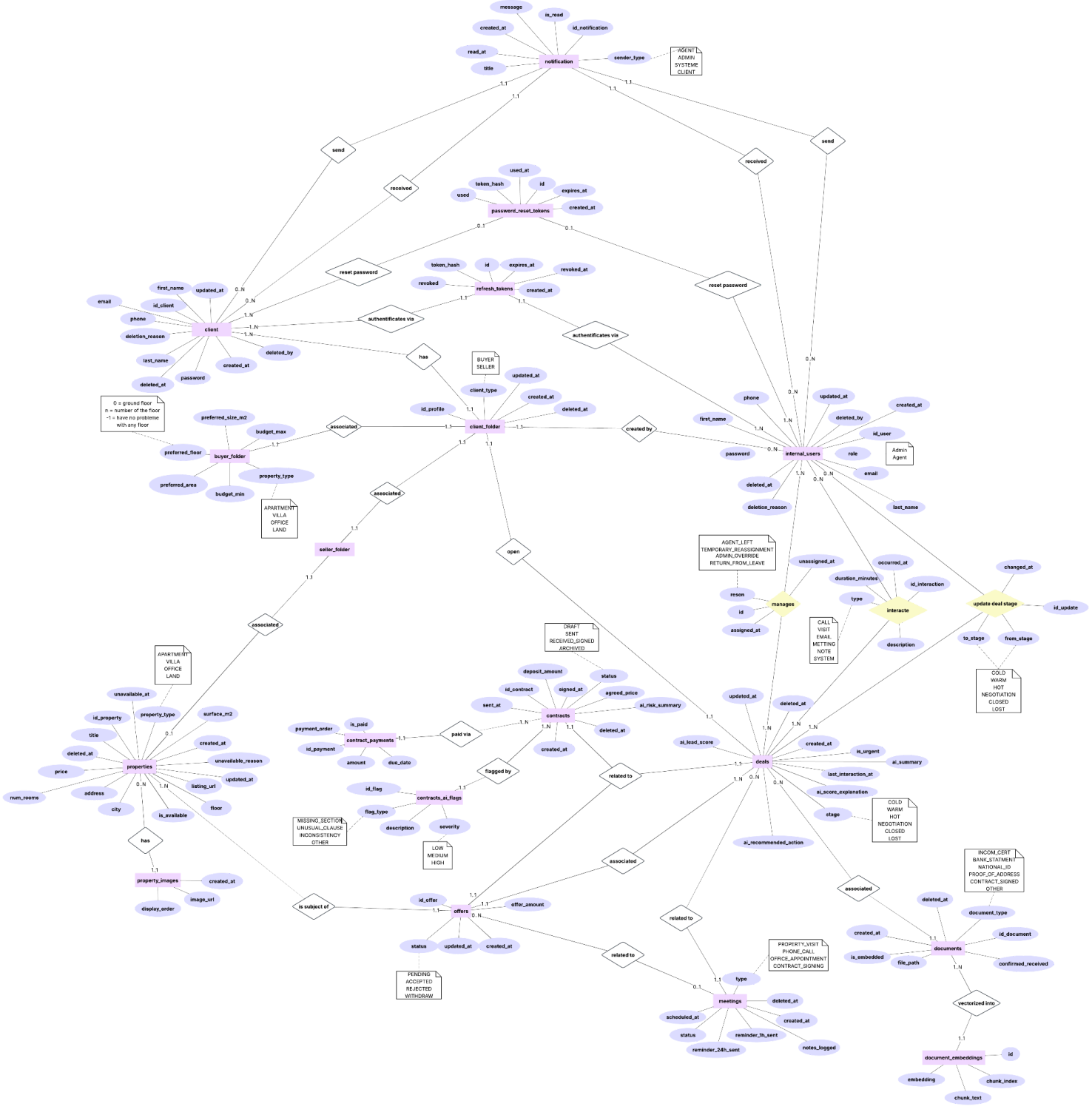
- **User (Abstract) :** Entité mère gérant l'identité (Google OAuth, Mot de passe). Se décline en des classes concrètes représentant les profils.
- **Client & Agent :** Séparation de l'information acteur (Auth) de l'information comportementale (CRM).
- **Dossier (Deal / Folder) :** Regroupe les opérations liant un client à son projet d'achat ou de vente.
- **Property (Biens) :** Les fiches descriptives des biens immobiliers stockées avec référence.
- **Contract :** Modèle du document légal représentant les offres, incluant les statuts d'acompte et d'accord (AgreedPrice).
- **Interaction / Meeting :** Modélise les points de contact entre l'agent et le client (appels, emails, visites).
- **Document :** Modélise tout fichier téléversé et stocké sur le cloud (Cloudinary), qu'il s'agisse de contrats, pièces d'identités ou images.



3.4 Modèle de données

Le schéma s'appuie sur PostgreSQL et est structuré autour du paradigme relationnel pour le transactionnel, adossé à pgvector pour la recherche sémantique des documents.

- **Architecture des Tables :** Les namespaces sont organisés, comme pour l'applicatif, sans foreign keys croisées invasives (les liaisons se limitent aux IDs pour préserver la portabilité).
- **Migration Flyway :** L'entièreté de la création de la base de données est gérée automatiquement par des scripts Flyway.
- **Contraintes Techniques :** Utilisation d'énumérations (@Enumerated(EnumType.STRING)) pour assurer la lisibilité en BDD sur les statuts (ex: MeetingStatus, DealStage, InteractionType) tout en contraignant l'intégrité à l'échelle de la JVM. L'utilisation de types VARCHAR avec contraintes au lieu de relations complexes allège les jointures temporelles.
- **Tables Principales Impliquées :** clients, agents, deals, properties, meetings, interactions, contracts, ai_document_embeddings.



3.5 Communication inter-modules

La synchronisation entre le module CRM, le Moteur IA et le module Notification est assurée par un bus applicatif natif Spring Events, permettant l'inversion de contrôle (Asynchrone/Publish-Subscribe).

➔ **Événements Publiés et Processus Déclenchés :**

- **CLIENT_CREATED :** (Publié par CRM Core) intercepté par Notification pour envoyer les URL d'accès (n8n), et par l'IA pour initialiser le profil du client dans l'espace RAG.
- **MEETING_SCHEDULED / MEETING_COMPLETED :** (Publié par CRM) Notification déclenche une alerte sortante (n8n). IA réceptionne l'événement COMPLETED pour déclencher une analyse et générer le résumé d'entretien (InteractionSummary).
- **CONTRACT_SENT :** L'écouteur NotificationEventListener intercepte le Payload et interroge le Webhook n8n pour délivrer le PDF de contrat (stocké sur Cloudinary) à l'email du client par SendGrid.
- **LEAD_STATUS_CHANGED :** Permet au module Notification de mettre à jour le système asynchrone si un deal devient bloqué.

Chapitre 4 - Choix technologiques

Les choix technologiques qui suivent reflètent trois priorités constantes tout au long du développement : la robustesse pour un contexte métier professionnel, la cohérence entre les couches pour minimiser la complexité opérationnelle, et la flexibilité pour faire évoluer la plateforme sans dette technique. Chaque technologie retenue répond à un besoin précis et s'intègre dans un écosystème pensé comme un tout.

4.1 Backend :

Le backend expose l'ensemble des fonctionnalités métier sous forme d'une API REST consommée par les **deux frontends**, tout en orchestrant les appels vers les services externes **(NVIDIA NIM, Cloudinary, SendGrid)** et en gérant le **pipeline RAG** de **l'intelligence artificielle**. Ce rôle central imposait des exigences fortes en termes de robustesse, de sécurité et de capacité à gérer des opérations asynchrones longues , notamment les appels aux modèles de langage qui peuvent durer plusieurs secondes.

4.1.1 Spring Boot 3.2.5 :

Spring Boot a été choisi pour sa maturité, sa stabilité et sa large adoption en entreprise. Il permet de structurer le backend en couches séparées (controllers, services et repositories), facilitant la maintenance et l'évolution du CRM. Sa compatibilité native avec Spring AI a également simplifié l'intégration des modèles d'intelligence artificielle NVIDIA NIM.

4.1.2 Java 21 :

Java 21 a été choisi pour ses **Virtual Threads**, qui améliorent la gestion des opérations concurrentes. Cette fonctionnalité permet de traiter efficacement les appels aux services d'**IA** et les tâches d'indexation **RAG** tout en réduisant la consommation des ressources système et en améliorant les performances de l'application.

4.1.3 Spring Security + JWT + Google OAuth2 :

La sécurité de Rawabet repose sur deux mécanismes d'authentification complémentaires : **JWT** pour les agents et administrateurs, garantissant une architecture stateless et facilement scalable, et **Google OAuth2** pour les clients afin de simplifier l'inscription et la connexion. Chaque requête protégée est contrôlée par un filtre de sécurité qui vérifie le token et identifie l'utilisateur, tandis que certains endpoints publics restent accessibles sans authentification.

4.1.4 Spring Data JPA / Hibernate :

Spring Data JPA et **Hibernate** ont été choisis pour simplifier la gestion des données relationnelles du CRM. Ils permettent de manipuler les entités métier (clients, dossiers, propriétés, rendez-vous, documents, etc.) à travers des repositories sans écrire systématiquement du SQL, tout en assurant la gestion automatique des relations entre les tables et l'optimisation des requêtes.

4.1.5 Apache PDFBox 2.0.31 :

Apache PDFBox a été choisi pour l'extraction du texte contenu dans les documents **PDF** déposés par les clients. Cette bibliothèque open-source permet de récupérer automatiquement le contenu des contrats, relevés bancaires ou justificatifs afin de l'exploiter dans le pipeline **RAG**. Les textes extraits sont ensuite transformés en vecteurs et stockés dans PostgreSQL sur Supabase (pgvector), permettant à l'intelligence artificielle d'analyser et d'interroger efficacement les documents du CRM.

4.1.6 Lombok 1.18.34 :

Lombok génère automatiquement à la compilation le code Java répétitif : constructeurs, getters, setters, builders et méthodes equals/hashCode. Dans Rawabet, toutes les entités JPA et les DTOs utilisent les annotations Lombok, réduisant significativement la taille du code source et améliorant sa lisibilité.

4.2 Frontend :

Le frontend de repose sur **deux applications** React distinctes : une dédiée aux **agents et administrateurs**, et une autre destinée aux **clients**. Cette séparation renforce la sécurité en isolant les fonctionnalités sensibles et permet de faire évoluer et maintenir chaque interface de manière indépendante.

4.2.1 React 18 + TypeScript 5.5.3 :

React 18 a été choisi pour sa popularité, son vaste écosystème et ses performances, offrant une interface utilisateur moderne et réactive. **TypeScript** complète cette technologie en apportant un typage statique qui améliore la qualité du code, facilite la maintenance et réduit les erreurs lors des échanges de données avec le backend. Cette combinaison garantit le développement d'une application fiable, évolutive et adaptée aux besoins du CRM.

4.2.2 Vite 5.4.1 :

Grâce à son fonctionnement basé sur les modules ES natifs, il offre un démarrage quasi instantané du serveur de développement ainsi qu'un rechargement à chaud **(Hot Module Replacement)** très performant.

4.2.3 TanStack Router 1.59.0 et TanStack Query 5.56.2 :

Ils assurent ensemble un CRM robuste et performant : le **Router** sécurise toutes les routes TypeScript et optimise le chargement via le lazy loading, tandis que **Query** gère le cache, les états serveur et la synchronisation automatique des données, garantissant des informations toujours à jour même en cas d'utilisation simultanée par plusieurs agents.

4.2.4 Radix UI + Tailwind CSS 3.4.4 :

Radix UI fournit des composants accessibles et non stylisés (ARIA-ready), tandis que Tailwind CSS gère entièrement le design via ses classes utilitaires.

4.2.5 React Hook Form + Zod :

React Hook Form optimise la gestion des formulaires en réduisant les re-renders, tandis que Zod assure une validation typée et centralisée avec inférence **TypeScript** et leur combinaison simplifie les formulaires complexes (comme les dossiers clients) en unifiant types et règles de validation dans un seul schéma, sans duplication.

4.2.6 Axios 1.16.1 :

Axios centralise les appels **HTTP** vers le backend **Spring Boot** et simplifie la gestion des requêtes dans les deux frontends. Grâce à ses intercepteurs, Rawabet injecte automatiquement le JWT dans chaque requête et gère globalement les erreurs 401 en redirigeant vers la page de login, évitant ainsi toute duplication de logique côté API.

4.2.7 react-markdown 10.1.0 :

react-markdown convertit le Markdown en HTML exploitable dans le navigateur. Dans Rawabet, il est essentiel pour le chatbot **MURSHID** et les réponses de l'IA, car il permet d'afficher correctement le texte structuré (titres, listes, gras) généré par les LLMs, évitant l'affichage brut de la syntaxe Markdown et améliorant ainsi l'expérience utilisateur.

4.3 Base de données

La base de données de Rawabet repose sur PostgreSQL avec l'extension pgvector, ce qui lui permet de gérer à la fois les données relationnelles du CRM et les vecteurs pour le RAG dans un seul système. Ce choix évite la complexité de deux bases séparées, simplifie l'architecture et réduit les coûts tout en supportant la logique métier et l'intelligence artificielle.

4.3.1 PostgreSQL :

PostgreSQL est utilisé comme base de données principale de Rawabet en raison de la nature fortement relationnelle des données CRM (clients, dossiers, agents, propriétés, rendez-vous). Il nous a permet de gérer efficacement les relations complexes et de garantir l'intégrité des données grâce aux jointures et contraintes référentielles, assurant ainsi une cohérence stricte et l'absence de données orphelines dans l'ensemble du schéma métier.

4.3.2 Supabase (AWS eu-north-1) :

Supabase est utilisé comme hébergeur PostgreSQL managé pour simplifier l'infrastructure et éviter la gestion directe du serveur en phase de développement. Il fournit des sauvegardes automatiques, du monitoring intégré et permet d'activer facilement pgvector pour le RAG de Rawabet. Hébergé sur AWS en Europe, il assure de bonnes performances et une conformité adaptée aux données sensibles du domaine immobilier.

4.3.3 pgvector :

Extension PostgreSQL qui permet de gérer les embeddings directement dans la base. Dans Rawabet, elle sert au **RAG** pour stocker les vecteurs des documents clients et retrouver les passages les plus pertinents via similarité cosinus, sans base vectorielle externe

4.3.4 HikariCP en Transaction Mode :

Pool de connexions Spring Boot configuré avec Supabase/PgBouncer pour optimiser les connexions limitées. Il permet de gérer la forte concurrence avec un petit pool tout en assurant la stabilité, avec une configuration adaptée.

4.4 Intelligence artificielle

L'intelligence artificielle est la proposition de valeur centrale de Rawabet. Elle n'est pas une fonctionnalité annexe elle est intégrée au cœur de trois processus métier essentiels : l'**analyse automatique des dossiers clients**, **assistant dans l'interface client , la recherche sémantique sur les documents uploadés, et l'onboarding conversationnel des nouveaux clients via le chatbot Murshid.** L'ensemble de cette couche IA repose sur Spring AI comme framework d'abstraction et NVIDIA NIM comme fournisseur de modèles de langage.

4.4.1 Spring AI 1.0.0-M1 :

Spring AI abstrait la communication avec les APIs de LLMs derrière des interfaces standardisées. Dans Rawabet, cette abstraction rend le code métier totalement indépendant du fournisseur , migrer de **NVIDIA NIM** vers un autre modèle ne nécessiterait qu'un changement de configuration.

4.4.2 NVIDIA NIM :

NVIDIA NIM a été retenu pour son API compatible OpenAI, ses modèles open-source performants (Llama 3.1 Nemotron 70B) avec quotas gratuits, et ses capacités de raisonnement structuré qui produisent des analyses de dossiers pertinentes et bien organisées.

4.4.3 Le système RAG :

Le RAG permet à l'IA de répondre à partir du contenu réel des documents clients. À l'upload, **PDFBox** extrait le texte, qui est découpé en chunks et vectorisé via NVIDIA NIM, puis stocké dans pgvector. À la question, les passages les plus proches sémantiquement sont récupérés et injectés comme contexte au LLM pour générer une réponse ancrée dans les documents réels.

4.4.4 Le chatbot Murshid :

Murshid guide les nouveaux clients dans la création de leur profil et dossier via une conversation étape par étape. À l'issue de l'échange, les données sont automatiquement persistées en base et un agent est assigné selon sa disponibilité et sa charge de travail.

4.4.5 Analyse automatique des dossiers :

Quand un agent demande une analyse, le pipeline RAG récupère les chunks pertinents des documents du client et les injecte dans un prompt structuré. Le LLM retourne une analyse couvrant la solvabilité, les risques identifiés et les recommandations , uniquement basée sur le contenu réel des documents, sans hallucination.

4.5 Automatisation

Rawabet intègre n8n pour orchestrer les processus métier complexes qui impliquent plusieurs systèmes sans nécessiter de développement backend supplémentaire. Le

backend Spring Boot déclenche les workflows via des appels HTTP vers les webhooks n8n, qui prend en charge la suite des actions.

4.5.1 n8n :

n8n est utilisé comme moteur d'automatisation open-source et auto-hébergeable, permettant de garder le contrôle des données sensibles. Il orchestre les principaux workflows CRM : onboarding clients et agents, gestion des réunions et rappels, suivi des leads inactifs, cycle de vie des contrats (envoi, relance, clôture, archivage), et génération de rapports automatiques, avec notifications et intégrations email/Calendar.

4.6 APIs externes

Rawabet s'appuie sur quatre services externes, chacun retenu pour répondre à un besoin précis que le développement en interne n'aurait pas justifié.

4.6.1 SendGrid :

SendGrid gère l'envoi des emails transactionnels : confirmations de compte, notifications de dossier et partage de rapports PDF. Il a été préféré à un SMTP direct pour sa réputation d'envoi qui garantit l'arrivée en boîte de réception plutôt qu'en spam , un détail critique pour les communications professionnelles d'une agence immobilière.

4.6.2 Google OAuth2:

Google OAuth2 permet aux clients de s'inscrire en deux clics sans créer de mot de passe supplémentaire. Ce choix réduit la friction à l'entrée de la plateforme tout en déléguant la sécurité de l'authentification à Google, réduisant ainsi la surface d'attaque de l'application.

4.6.3 Cloudinary :

Cloudinary gère le stockage et la distribution CDN des fichiers media , photos de biens immobiliers et pièces justificatives clients. Ses transformations d'images à la volée permettent de servir chaque photo dans le format et la résolution optimaux selon le device, sans stocker de multiples versions du même fichier.

4.6.4 RapidAPI , Realty in US + Zillow

Ces deux APIs enrichissent les fiches propriétés avec des données de marché externes : annonces comparables, historique des prix et estimations de valeur via Zillow. Elles donnent aux agents un avantage informationnel concret lors des négociations avec les clients.

Chapitre 5 - Implémentation

5.1 Module Auth

Le module **Auth** est le gardien centralisé de l'identité dans Rawabet. Il gère l'authentification de deux populations distinctes : agents/administrateurs et clients , via deux portails séparés partageant la même infrastructure de sécurité.

5.1.1 Dual-Portal :

Rawabet distingue deux portails d'authentification identifiés par un paramètre portal. Les **agents et administrateurs** s'authentifient via le **portail ADMIN_AGENT**, les clients via le **portail CLIENT**. À chaque connexion, le service vérifie que le compte est actif (deletedAt == null), compare le mot de passe avec **BCrypt**, révoque tous les refresh tokens existants, puis génère une nouvelle paire access token et refresh token. Cette séparation garantit qu'un client ne peut jamais accéder aux fonctionnalités internes de l'agence, même avec des identifiants valides.

5.1.2 Gestion des tokens JWT :

Chaque requête HTTP entrante est interceptée par le **JwtAuthenticationFilter** qui lit le token depuis l'en-tête Authorization ou le cookie HTTP-only en cas d'absence, valide la signature et l'expiration, extrait l'email, charge l'utilisateur en base et l'injecte dans le contexte de sécurité Spring. Le mécanisme de refresh implémente la rotation : à chaque renouvellement, l'ancien token est révoqué et un nouveau est émis, garantissant qu'un token volé est invalidé dès l'utilisation légitime suivante.

5.1.3 Google OAuth2

L'intégration Google OAuth2 permet aux utilisateurs de se connecter avec leur **compte Google**. Le frontend obtient un **id_token** Google via le bouton Sign-In et l'envoie au backend, qui le vérifie via **GoogleIdTokenVerifier**. La connexion Google ne fonctionne que si le compte a été préalablement lié via l'endpoint de liaison ,un compte non lié ne peut pas se connecter par ce biais, évitant toute usurpation d'identité par email.

5.1.4 Réinitialisation de mot de passe

Quand un utilisateur demande une réinitialisation, un token opaque est généré, haché en **SHA-256** et stocké en base. Seul le token brut est envoyé par email via **SendGrid** (automatisé). Lors de la réinitialisation, le backend recalcule le hash et le compare à celui stocké, vérifie que le token n'a pas expiré (2 heures) et n'a pas déjà été utilisé, puis met à jour le mot de passe et marque le token comme utilisé pour prévenir toute réutilisation.

5.2 Module CRM Core

Le module CRM Core est le cœur métier. Il modélise l'ensemble du cycle de vie d'un dossier immobilier autour de **30 entités JPA**, depuis la création du profil client jusqu'à la clôture du deal, en passant par la gestion des propriétés, offres, contrats, interactions et rendez-vous.

5.2.1. Pipeline immobilier en 7 étapes :

Chaque dossier progresse à travers un pipeline structuré en 7 étapes : **NEW, COLD, WARM, HOT, NEGOTIATION, CLOSED_WON et CLOSED_LOST**. Cette modélisation permet aux agents de visualiser précisément où en est chaque prospect, et au moteur IA de calculer un score de lead et de suggérer l'étape suivante la plus appropriée selon le contexte du dossier.

5.2.2 Gestion des clients et dossiers :

La création d'un client se fait en **deux étapes** :

l'agent saisit d'abord les informations personnelles, puis complète le profil immobilier selon le type de projet. Les **dossiers acheteurs** stockent les critères de recherche (budget min/max, zone, superficie, étage préféré) dans **buyer_folder** , tandis que les **dossiers vendeurs** sont reliés aux propriétés réelles via **seller_folder** et **properties**. La suppression d'un client est toujours un **soft-delete** , les données sont conservées, seul deletedAt est renseigné.

5.2.3 Gestion documentaire

Les documents uploadés par les clients sont envoyés vers Cloudinary via **CloudinaryService**. L'URL retournée est stockée dans l'entité Document avec son type (DocumentType enum). Après chaque upload, un événement Spring déclenche l'indexation asynchrone par le moteur IA : **extraction du texte PDF, découpage en chunks, vectorisation et stockage dans pgvector**.

5.2.4 Gestion des offres et contrats :

Les offres d'achat sont créées et suivies via **OfferController**, avec comparaison automatique au prix demandé. Les contrats sont générés en PDF via Flying Saucer, uploadés sur Cloudinary, puis soumis automatiquement à l'audit de risque IA qui analyse les clauses critiques et stocke le résumé dans **Contract.aiRiskSummary**, visible directement dans le portail client.

5.2.5 Architecture événementielle :

Le module CRM publie neuf événements Spring **(AgentCreatedEvent, ClientCreatedEvent, MeetingScheduledEvent, ContractSignedEvent, etc.)** sans aucun couplage direct avec les modules qui les consomment. Le module Notification écoute ces événements pour déclencher les workflows n8n, et le module IA les intercepte pour recalculer les scores et indexer les documents. Cette architecture garantit que l'ajout d'un nouveau comportement ne nécessite jamais de modifier le code métier existant.

5.3 Module Moteur IA

Le module IA est le cerveau analytique de Rawabet. Il regroupe six services spécialisés qui automatisent les tâches à haute valeur ajoutée : scoring des leads, analyse RAG des documents, audit de contrats, résumé des interactions, suggestion d'étape de pipeline, recommandation de biens et génération de rapports hebdomadaires.

5.3.1 Lead Scoring

Le service de scoring calcule automatiquement un score de 0 à 100 pour chaque dossier, reflétant la probabilité de closing et l'urgence du lead. Il analyse l'historique complet des interactions, les rendez-vous effectués, le profil client et l'étape actuelle du pipeline. Le score, l'explication, l'action recommandée et le flag d'urgence sont persistés directement sur l'entité Deal et recalculés automatiquement à chaque événement significatif , nouvelle interaction, rendez-vous, changement d'étape.

5.3.2 Système RAG et assistant Murshid

Le pipeline RAG transforme les documents clients en connaissances interrogeables. À l'upload, PDFBox extrait le texte, découpé en chunks de 1000 caractères avec 200 caractères de chevauchement, puis vectorisé via l'API d'embedding NVIDIA NIM et stocké dans pgvector. À la question, l'embedding de la requête est comparé par similarité cosinus aux vecteurs stockés, les passages les plus proches sont récupérés et injectés comme contexte au LLM. L'assistant Murshid exploite ce pipeline pour répondre aux questions des clients sur leurs dossiers réels, avec deux modes disponibles : ciblé sur un dossier spécifique ou global sur tous les documents du client.

5.3.3 Audit de contrats et suggestions

Lors de la création d'un contrat, le moteur IA analyse automatiquement les clauses critiques (suspensives, pénalités, résiliation) et stocke le résumé de risques dans **Contract.aiRiskSummary**. En parallèle, le service de suggestion d'étape analyse le contexte complet du dossier et propose l'étape suivante la plus appropriée avec un niveau de confiance, tandis que le service de recommandation de biens compare les critères d'un acheteur aux propriétés disponibles et retourne une liste de correspondances scorées.

5.3.4 Rapport hebdomadaire

Chaque lundi à 08h00, déclenché par un cron n8n, le service agrège les données de la semaine écoulée, génère un rapport structuré via le LLM, produit un PDF via Flying Saucer et le retourne à n8n pour envoi à l'administrateur. Ce rapport couvre la progression des dossiers, les leads froids identifiés, les performances des agents et les actions recommandées.

5.4 Module Notification

Le module Notification orchestre l'ensemble des communications sortantes de Rawabet : emails transactionnels via SendGrid, notifications in-app et workflows n8n , en écoutant les événements publiés par le module CRM sans aucun couplage direct avec lui.

5.4.1. Architecture événementielle

Le composant **NotificationEventListener** écoute les neuf événements Spring du module CRM via **@EventListener** et les mappe vers leurs workflows n8n respectifs. Aucune dépendance directe n'existe entre les deux modules , le CRM publie, la Notification écoute et agit. Cette séparation permet d'ajouter de nouveaux canaux de communication (SMS, push mobile) sans toucher au code métier.

5.4.2 Service Webhook n8n

N8nWebhookService est le pont unique entre Spring Boot et n8n. Pour chaque événement, il construit un payload JSON standardisé contenant le type d'événement, les données métier et un horodatage, puis l'envoie en HTTP POST vers le chemin webhook n8n correspondant. Les erreurs sont gérées silencieusement , un échec de notification ne perturbe jamais le flux métier principal. L'URL de base n8n est centralisée dans application.yml et surchargeable par variable d'environnement pour faciliter le passage en production.

5.4.3 Emails et notifications in-app

Pour les communications directes sans passer par n8n, EmailService utilise le SDK SendGrid pour l'envoi des emails de réinitialisation de mot de passe et des communications ponctuelles depuis l'interface d'administration. En parallèle, InAppNotificationService gère les notifications affichées dans le portail client , badge de notification, centre de notifications avec statut lu/non-lu , accessibles depuis n'importe quelle page du portail via trois endpoints dédiés.

5.5 Portail Client

Le portail client est une application React indépendante (`frontend-client`), distincte du portail d'agent et d'admin. Cette séparation garantit que le client n'a accès qu'à ses propres données et qu'aucune information interne de l'agence n'est exposée.

5.5.1 Landing Page et Acquisition

Avant toute connexion, le portail présente une page publique de présentation des services de l'agence. Un chatbot d'inscription conversationnel est prévu pour remplacer ce formulaire traditionnel en collectant les informations du prospect (nom, projet, coordonnées) de manière interactive. Si un utilisateur déjà authentifié accède à cette page publique, il est automatiquement redirigé vers son espace client.

5.5.2 Authentification et Gestion de Session

Le client ne crée pas son compte lui-même. C'est l'agent qui crée le profil client, ce qui déclenche automatiquement la génération d'un compte avec des identifiants provisoires (email et mot de passe temporaire) envoyés au client par email via SendGrid.

Lors de sa première connexion, le client est invité à changer son mot de passe avant d'accéder au portail. Cette étape garantit que les identifiants provisoires ne restent pas en usage.

Après validation des identifiants, le système stocke le token JWT et l'identifiant client (`client_id`) en local, puis redirige automatiquement le client vers son tableau de bord. Toutes les routes du portail (`/client/*`) sont protégées : chaque requête envoyée au backend inclut automatiquement le token dans son en-tête d'autorisation via un intercepteur Axios. Un client non authentifié est systématiquement renvoyé vers la page de connexion.

5.5.3 Tableau de Bord (Accueil)

La page d'accueil offre une vue d'ensemble dynamique de la situation immobilière du client. Elle est structurée autour de plusieurs blocs :

Indicateurs financiers : deux métriques sont calculées et affichées automatiquement - la capacité d'investissement (cumul des budgets pour les dossiers d'achat) et la valeur du patrimoine en vente (valeur totale des biens mis en vente par le client).

Analyses graphiques : des graphiques en secteurs et en barres visualisent la répartition des projets (Achat vs Vente) ainsi que l'avancement des dossiers par étape du pipeline.

Liste des dossiers actifs : chaque dossier est présenté sous forme de carte détaillée affichant le score IA (Lead Score) évaluant la qualité et l'avancement du dossier, une barre de progression indiquant le pourcentage de complétion du projet, ainsi que les informations clés : type de projet (Achat ou Vente), budget, ville et agent dédié.

Widgets de suivi rapide : trois widgets permettent au client d'avoir une vision immédiate des points d'attention sans naviguer dans le portail - le prochain rendez-vous (type, date, heure et lieu) avec un accès direct à la gestion des rendez-vous, un rappel visuel des pièces justificatives manquantes qui bloquent l'avancement des dossiers, et un flux d'activité récente résumant les dernières actions effectuées sur ses dossiers.

Équipe dédiée : le client voit l'ensemble des agents travaillant sur ses différents dossiers, avec une approche multi-agent permettant à plusieurs commerciaux d'intervenir sur un même profil.

5.5.4 Gestion des Dossiers

Le portail répertorie l'ensemble des dossiers immobiliers du client (achat et vente) sur une page dédiée. Un comportement d'auto-redirection est implémenté : si le client ne possède qu'un seul dossier actif, il est automatiquement redirigé vers sa vue détaillée sans passer par la liste.

Vue en liste : chaque dossier est présenté sous forme de carte synthétique affichant les informations essentielles - le type de bien (Appartement, Villa, etc.), la date de création, l'agent assigné et un badge "Urgent" si le dossier nécessite une attention immédiate. Trois compteurs de ressources complètent chaque carte en indiquant le nombre de propriétés associées, de documents fournis et de rendez-vous effectués. Un indicateur circulaire affiche le score IA du dossier pour permettre au client de prioriser ses actions en un coup d'œil.

Le client peut affiner l'affichage grâce à des filtres par type de projet (achat ou vente), par agent responsable et par état d'avancement.

Vue détaillée : la page de détail est une vue 360° dédiée à un dossier spécifique, organisée en plusieurs blocs :

- **Identification et contact** : rappel des coordonnées de l'agent responsable avec un bouton d'appel direct et le badge d'urgence si applicable.
- **Barre de progression** : visualisation linéaire des 7 étapes clés du parcours immobilier, du statut "Nouveau" jusqu'à la "Clôture", permettant au client de situer précisément l'avancement de son projet.
- **Caractéristiques du bien** : détail technique complet (surface, étage, type, quartier, nombre de pièces) et informations financières (budget pour un achat ou prix demandé pour une vente). Pour les dossiers de vente, une galerie photos du bien mis en marché est également disponible.
- **Rendez-vous spécifiques** : liste chronologique des rendez-vous liés uniquement à ce dossier, distincte du calendrier global du client.
- **Activité récente** : un flux chronologique condensé affiche les 5 derniers événements marquants du dossier. Six types d'événements sont identifiés visuellement : visites, signatures de contrats, appels, emails, documents ajoutés et offres soumises. Chaque action est accompagnée d'une icône thématique et d'une date précise, reliées entre elles par une ligne de temps verticale. Un lien direct permet au client de basculer vers la page de chronologie complète pour consulter l'intégralité de l'historique du dossier.
- **Onglets de gestion** : la partie inférieure de la page est organisée en trois onglets distincts :
    - ***Mes Offres*** : historique des offres soumises pour les acheteurs, avec comparaison au prix demandé, analyse de l'écart et boutons pour accepter ou refuser les contre-offres reçues.
    - ***Mes Contrats*** : liste des documents contractuels (compromis de vente, mandats) avec le détail financier associé (prix convenu, montant du dépôt de garantie) et une analyse IA des risques pour chaque contrat.
    - ***Documents Administratifs*** : liste de contrôle des pièces justificatives requises avec un système d'upload direct intégré et la visualisation des documents déjà validés par l'agence. Une confirmation visuelle (toast) s'affiche immédiatement après chaque téléversement réussi.

5.5.5 Gestion Documentaire et Contrats

Le portail dispose d'un espace de stockage et de gestion documentaire global, organisé pour offrir au client une visibilité complète sur l'ensemble de ses pièces justificatives et engagements contractuels.

Organisation par dossier : tous les documents sont regroupés par projet immobilier pour une meilleure clarté. Un menu déroulant de navigation permet au client de se focaliser sur les documents d'un dossier précis sans afficher l'ensemble du catalogue.

Pièces administratives : le suivi des documents personnels (CIN, justificatifs de revenus, etc.) est géré via une liste de contrôle avec un indicateur de statut par document - "À fournir", "Complété" si le document a été reçu et validé par l'agence.

Espace contrats : une section dédiée regroupe les engagements officiels du client (compromis de vente, mandats) avec leur statut de signature affiché clairement (Signé ou En attente). Chaque contrat bénéficie d'une analyse de risque IA qui résume les clauses critiques pour sécuriser le client, et peut être consulté directement via une visionneuse PDF/Image intégrée sans téléchargement préalable.

5.5.6 Gestion des Rendez-vous

Le client dispose d'une interface complète pour gérer ses rendez-vous:

- **Rappels Prioritaires** : Mise en avant des rendez-vous imminents (moins de 24h ou moins de 1h) avec alertes visuelles.
- **Typologie des RDV** : Identification visuelle des types de rencontres (Visite de bien, Appel, Signature de contrat, RDV Agence).
- **Gestion des Actions** : Le client a la possibilité d'accepter une proposition de rendez-vous directement depuis le portail. Pour toute demande de reprogrammation ou d'annulation, il doit contacter directement son agent responsable.
- **Détails Logistiques** : Affiche la date, l'heure, le lieu exact et les notes de préparation laissées par l'agent.

5.5.7 Assistant IA - MURSHID

Le portail intègre un assistant conversationnel nommé **MURSHID**, basé sur la technologie RAG (Retrieval-Augmented Generation). Il consulte les données réelles du client en temps réel pour formuler ses réponses, ce qui lui permet de répondre à des questions précises sur l'avancement du dossier, les rendez-vous à venir ou les contrats en cours.

L'assistant est accessible depuis un widget flottant disponible sur toutes les pages du portail. Deux modes de conversation sont disponibles selon le contexte - un mode global et un mode lié à un dossier spécifique. Chaque réponse affiche les sources de données utilisées pour la justifier. Des suggestions de questions rapides sont également proposées pour guider les clients moins à l'aise avec l'outil.

5.5.8 Espace Profil et Sécurité

Le client dispose d'un espace profil dédié depuis lequel il peut consulter ses informations personnelles et les modifier partiellement. Le nom et prénom sont affichés en lecture seule

afin de garantir l'intégrité juridique des données liées aux contrats. Seules les coordonnées de contact - l'adresse email et le numéro de téléphone - sont modifiables par le client. Une section sécurité complète cet espace, permettant la modification du mot de passe.

5.5.9 Centre de Notifications

Un centre de notifications interactif informe le client en temps réel des nouvelles actions requises de sa part. Les notifications sont consultables depuis n'importe quelle page du portail.

5.6 Automatisations n8n

5.6.1 Principe d'intégration

n8n est exécuté localement via la commande `npx n8n` et est accessible sur `http://localhost:5678`. Son intégration avec Spring Boot repose sur un mécanisme de webhooks HTTP : c'est toujours le backend qui prend l'initiative d'appeler n8n, et non l'inverse.

Un service dédié, `N8nWebhookService`, centralise toute la logique d'envoi. Il assemble pour chaque événement un payload JSON standardisé contenant le type d'événement, les données métier associées et un horodatage, puis le poste vers le chemin webhook n8n correspondant. L'URL de base de n8n est centralisée dans le fichier de configuration (`application.yml`) et peut être surchargée via une variable d'environnement, ce qui facilite le passage d'un environnement local à un environnement de production sans modification du code.

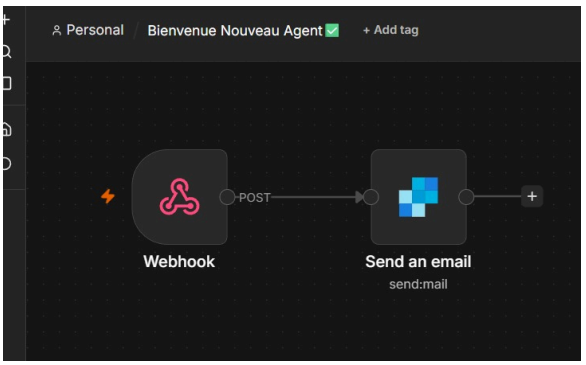
Les appels vers n8n sont déclenchés par le système d'événements interne de Spring. Chaque action métier importante publie un événement dans le contexte de l'application, et le composant `NotificationEventListener` écoute ces événements pour appeler le service webhook avec les données appropriées.

5.6.2 Workflows implémentés

Huit workflows sont implémentés, couvrant l'ensemble du cycle de vie d'un dossier immobilier.

Workflow 1 - Création d'un agent

Lorsqu'un administrateur crée un nouveau compte agent dans le CRM, le backend envoie un webhook vers `/webhook/agent-cree` contenant l'identifiant de l'agent, ses nom et prénom, son adresse email, son mot de passe temporaire et un lien d'activation. n8n se charge d'envoyer l'email de bienvenue à l'agent avec ses identifiants provisoires.

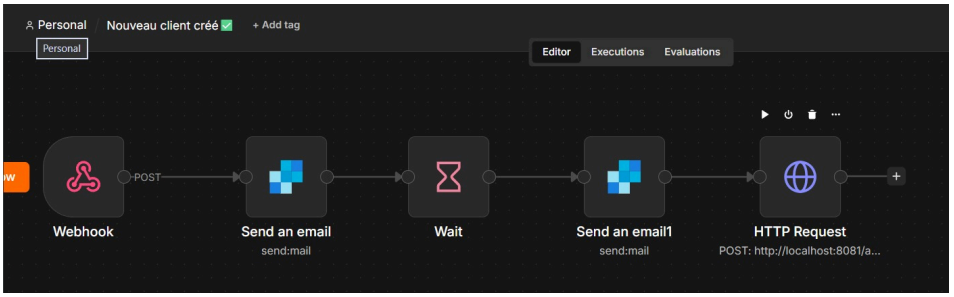


Workflow 2 - Création d'un client

Lors de la création d'un nouveau profil client par un agent, un webhook est envoyé vers `/webhook/nouveau-client` avec l'identifiant du client, ses nom et prénom, son email et son mot de passe temporaire. n8n enchaîne alors les actions suivantes :

1. Envoie immédiatement un email de bienvenue au client avec ses identifiants d'accès au portail via SendGrid.
2. Attend 3 jours et envoie un email de suivi au client pour s'assurer qu'il a bien pris en main son espace et pour répondre à d'éventuelles questions.
3. Journalise les deux envois d'emails dans le CRM via un callback vers `/api/webhooks/log-email`, assurant ainsi la traçabilité complète des communications dans l'historique du dossier client.

Ce workflow illustre la capacité de n8n à gérer des séquences d'actions différées dans le temps sans aucune intervention manuelle de l'équipe.



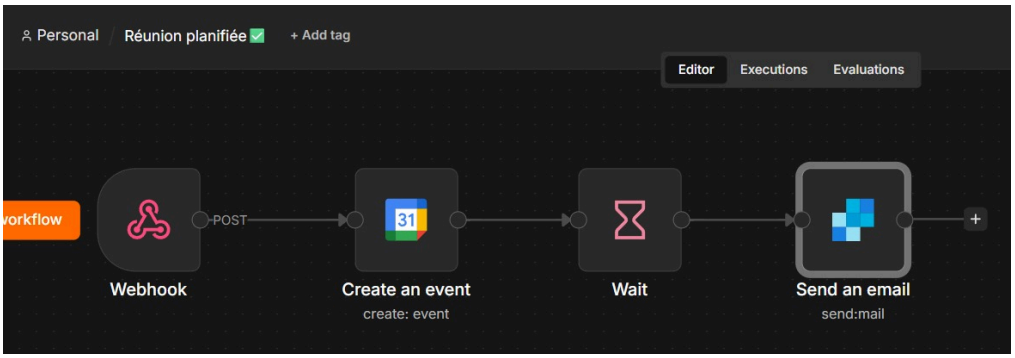
Workflow 3 - Réunion planifiée

Dès qu'un agent planifie une réunion avec un client, le backend notifie n8n via `/webhook/reunion-planifiee` en transmettant l'identifiant de la réunion, la date et l'heure prévues, le type de réunion ainsi que les coordonnées du client et de l'agent. n8n enchaîne alors les actions suivantes :

1. Crée automatiquement un événement Google Calendar avec le titre du rendez-vous, la date, l'heure et le lieu, et envoie une invitation à l'agent et au client via leurs adresses email respectives. Les deux parties reçoivent ainsi une invitation standard sur tous leurs appareils.

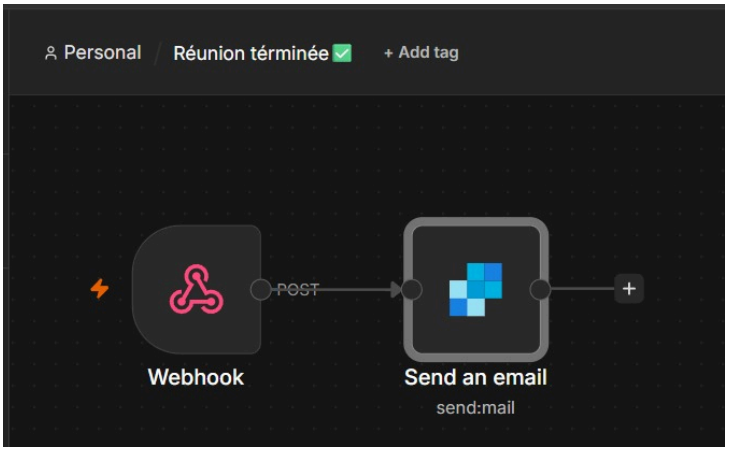
2. Attend jusqu'à 24 heures avant l'heure prévue du rendez-vous et envoie un email de rappel à l'agent et au client pour s'assurer qu'aucune des deux parties n'oublie le rendez-vous.

Cette intégration avec Google Calendar est gérée nativement par n8n via son nœud dédié, sans nécessiter de développement personnalisé côté backend.



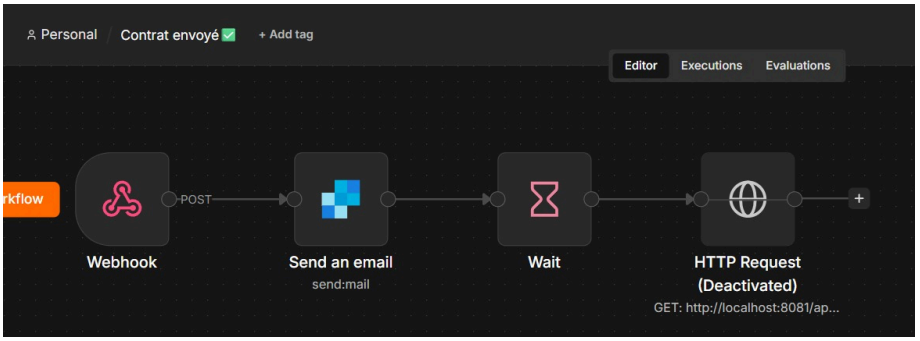
Workflow 4 - Rendez-vous terminée

Lorsqu'un rendez-vous est marquée comme terminée dans le CRM, un webhook est envoyé vers `/webhook/reunion-terminee` avec un sujet formaté (type de rendez-vous, nom du client, adresse du bien), l'identifiant du dossier et les coordonnées des deux parties. n8n déclenche une relance de suivi post-visite à destination de l'agent pour lui demander de journaliser ses notes et impressions sur la rendez-vous.



Workflow 5 - Contrat envoyé

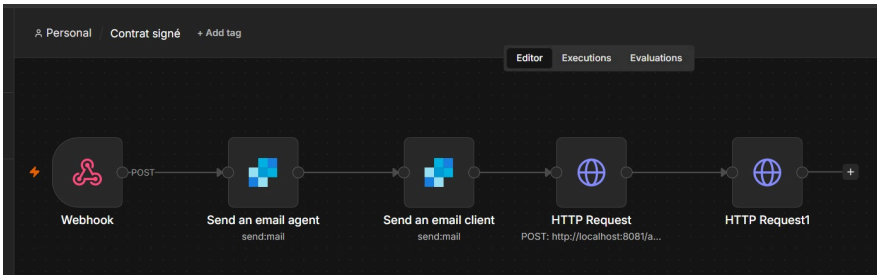
Lorsqu'un contrat est généré et transmis au client pour signature, le backend appelle `/webhook/contrat-envoye` en fournissant l'identifiant du contrat, l'URL du PDF, les coordonnées du client et de l'agent ainsi que l'identifiant du dossier. n8n notifie le client par email avec un lien direct vers son contrat.



Workflow 6 - Contrat signé

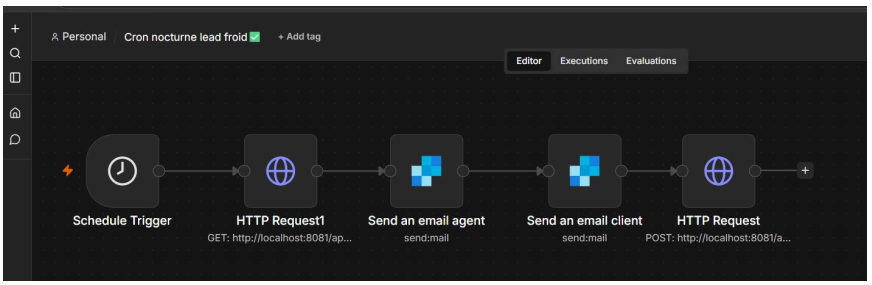
Lorsqu'un client signe son contrat, le backend notifie n8n via `/webhook/contrat-signe` avec l'identifiant du contrat, du deal et du dossier, ainsi que les coordonnées du client et l'email de l'administrateur principal. n8n enchaîne alors les actions suivantes :

1. Envoie un email de félicitations au client pour confirmer la bonne réception de son contrat signé et le féliciter pour l'aboutissement de son projet immobilier.
2. Envoie une notification interne à l'administrateur pour l'informer qu'un contrat a été signé et qu'un dossier est prêt à être clôturé.
3. Déclenche la clôture automatique du dossier dans le CRM via un callback vers l'endpoint `/api/webhooks/dossiers/{id}/cloturer`, qui met à jour le statut du dossier en base de données sans intervention manuelle.
4. Archive le PDF du contrat signé pour assurer la traçabilité et la conservation des documents légaux.



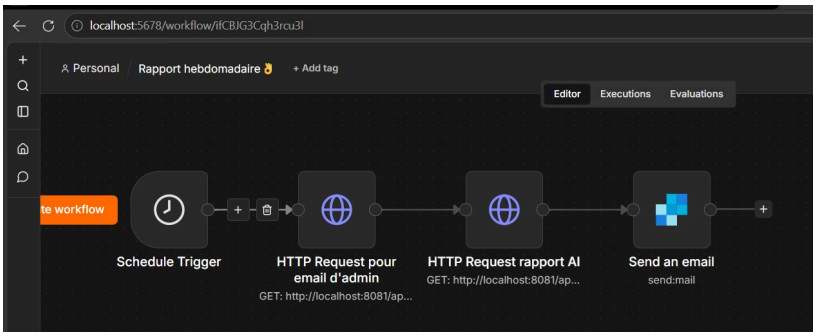
Workflow 7 - Leads froids

Contrairement aux workflows précédents, celui-ci n'est pas déclenché par un événement Spring Boot mais par un **cron job nocturne configuré directement dans n8n**. Chaque nuit, n8n appelle automatiquement l'endpoint `/api/webhooks/leads/cold` du backend, qui retourne la liste des dossiers n'ayant enregistré aucune activité depuis plus de 10 jours. Pour chaque dossier identifié comme froid, n8n envoie à l'agent responsable un email de relance ainsi qu'une notification in-app via endpoint `/api/webhooks/notifications/agent`, rappelant le nom du client et l'action recommandée.



Workflow 8 - Rapport hebdomadaire

Ce workflow est également déclenché par un **cron job configuré dans n8n**, programmé chaque lundi à 08h00. n8n appelle l'endpoint `/api/webhooks/ai-weekly-summary` du backend, qui déclenche la génération du rapport par le Moteur IA et retourne le fichier PDF en réponse. n8n récupère ce PDF et l'envoie par email à l'administrateur principal, dont l'adresse est obtenue via l'endpoint `/api/webhooks/admin/primary-email`. Le rapport couvre la progression des dossiers de la semaine écoulée, les leads froids identifiés, la comparaison des performances des agents et les actions recommandées.



5.6.3 Endpoints exposés par Spring Boot pour n8n

La communication est bidirectionnelle : n8n peut également rappeler le backend via des endpoints dédiés, regroupés sous le préfixe `/api/webhooks`. Ces endpoints sont exemptés de la vérification JWT afin de permettre à n8n de les appeler librement.

Méthode	Chemin	But
`GET`	`/api/webhooks/ai-weekly-summary`	n8n demande la génération du rapport AI hebdomadaire au format PDF. Le backend génère et retourne le fichier PDF en réponse.
`GET`	`/api/webhooks/leads/cold`	n8n demande la liste des leads "froids" (dossiers sans activité depuis plus de 10 jours). Le backend retourne la liste pour que n8n puisse envoyer des relances automatisées.
`GET`	`/api/webhooks/admin/primary-email`	n8n demande l'adresse email du premier administrateur enregistré dans le système, pour pouvoir lui envoyer des rapports ou notifications.
`POST`	`/api/webhooks/notifications/agent`	n8n envoie une notification in-app à un agent spécifique dans le CRM (identifié par son email). Le corps de la requête contient le destinataire, le titre et le message.
`POST`	`/api/webhooks/notifications/admin`	n8n envoie une notification in-app à un administrateur. Même fonctionnement que l'endpoint agent.
`POST`	`/api/webhooks/dossiers/{id}/cloturer`	n8n demande la clôture d'un dossier/deal spécifique (identifié par son UUID). Le backend exécute la clôture en base de données.
`POST`	`/api/webhooks/log-email`	n8n notifie le backend qu'un email a été envoyé, pour traçabilité. Le backend enregistre l'information dans ses logs.
`POST`	`/api/webhooks/generate-draft`	n8n soumet un prompt et un contexte pour demander au backend de générer un brouillon d'email via l'IA. Le backend retourne le texte généré.

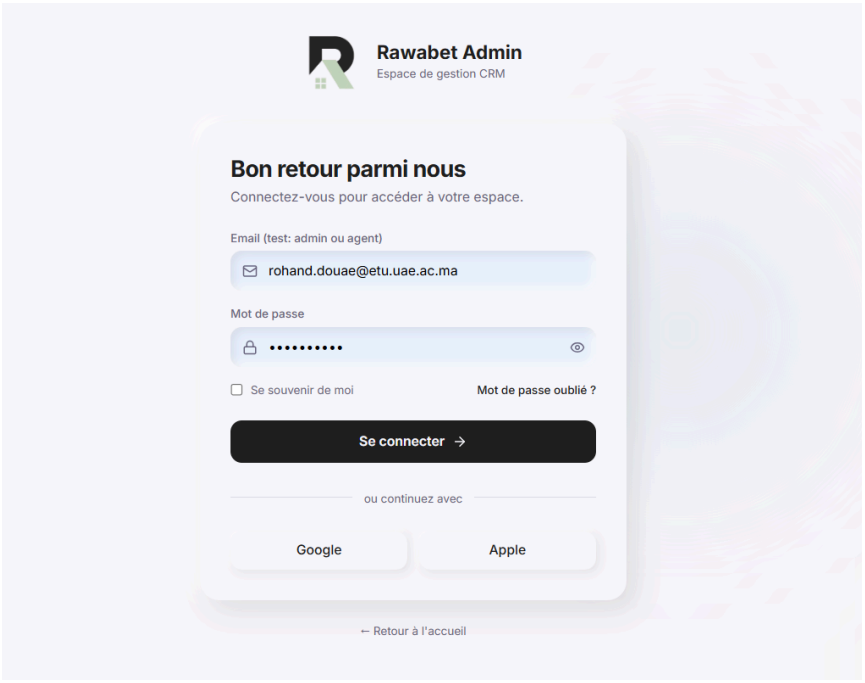
5.6.1 Sécurité des webhooks

Les endpoints exposés par Spring Boot pour n8n (`/api/webhooks/*`) sont explicitement exemptés de la vérification du token JWT dans la configuration Spring Security. Cette exemption est volontaire et limitée à ces seuls endpoints : n8n s'exécutant localement sur le même réseau que le backend, le risque d'accès non autorisé est maîtrisé dans le contexte du projet. Dans un environnement de production, ces endpoints seraient sécurisés par une clé secrète partagée transmise dans l'en-tête de chaque requête n8n.

5.7 Captures d'écran

5.7.1 Module Auth

- **Page de connexion**

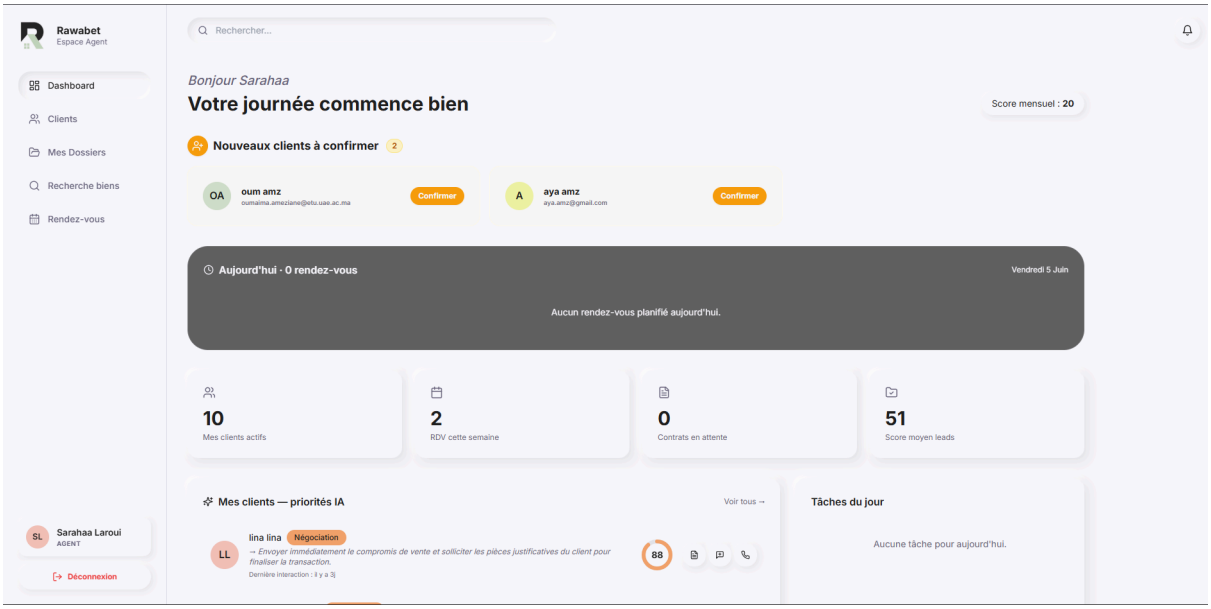


5.7.2 Landing page d'agent / Admin

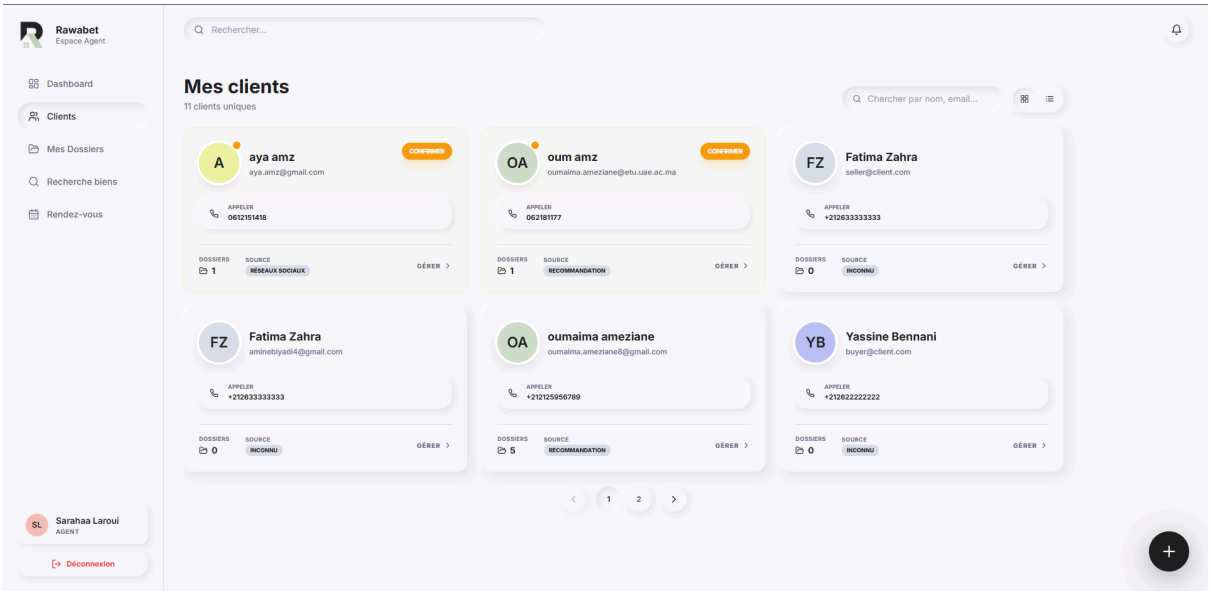


5.7.3 Espace Agent

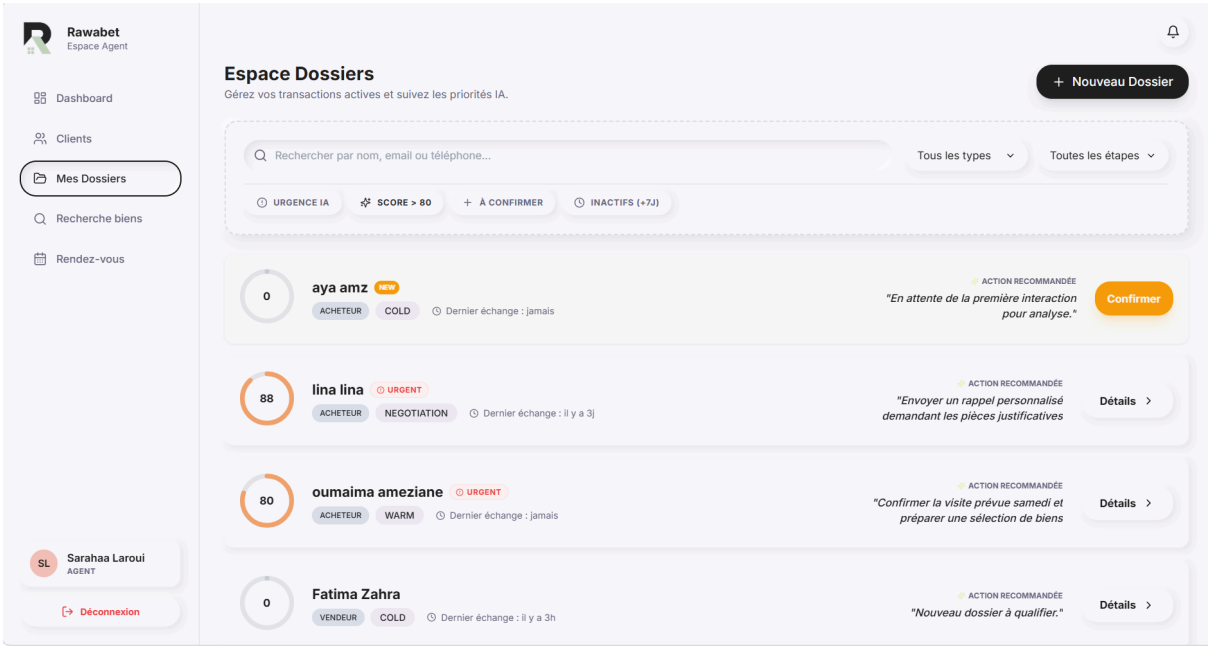
- **Dashboard agent (vue globale avec pipeline et indicateurs)**



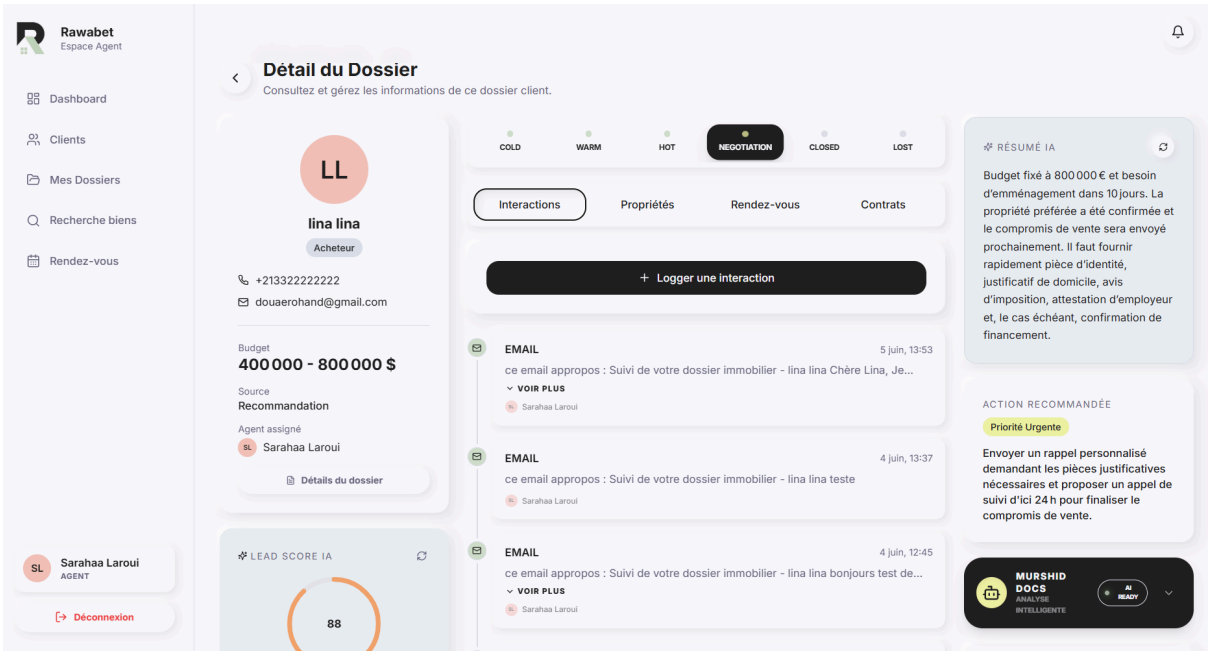
- **Liste des clients avec score IA et badges d'urgence**



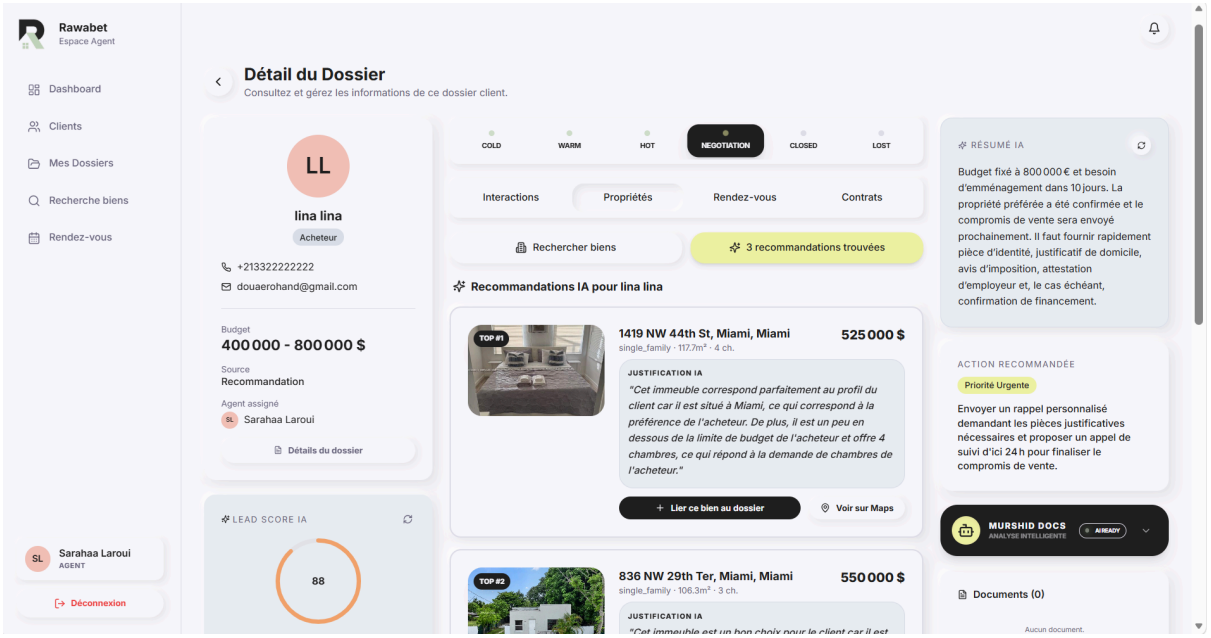
- **Dossiers clients**



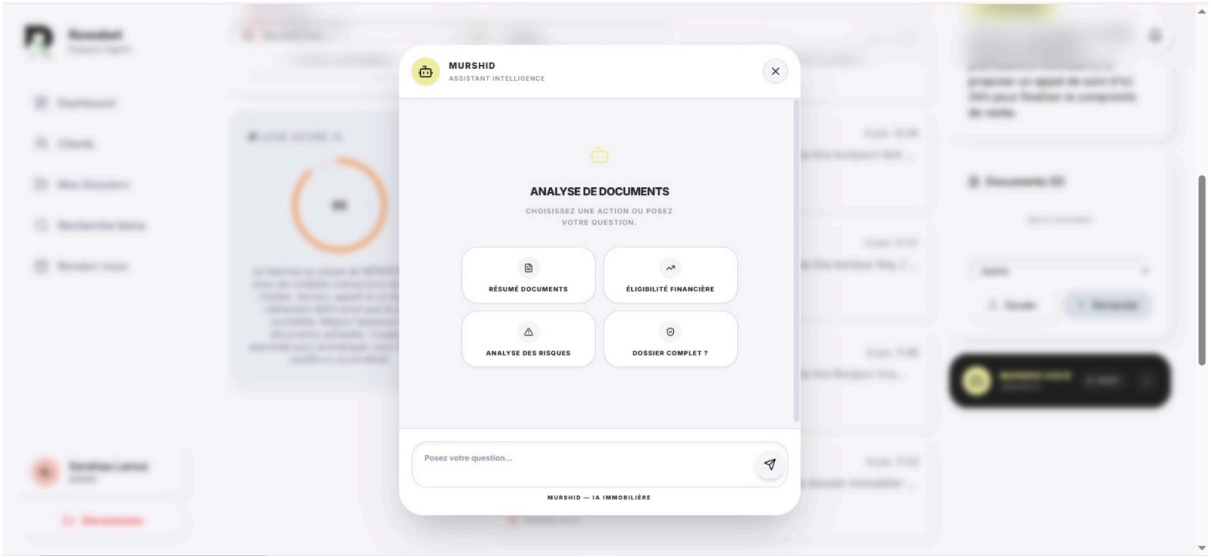
- **Détails du dossier**



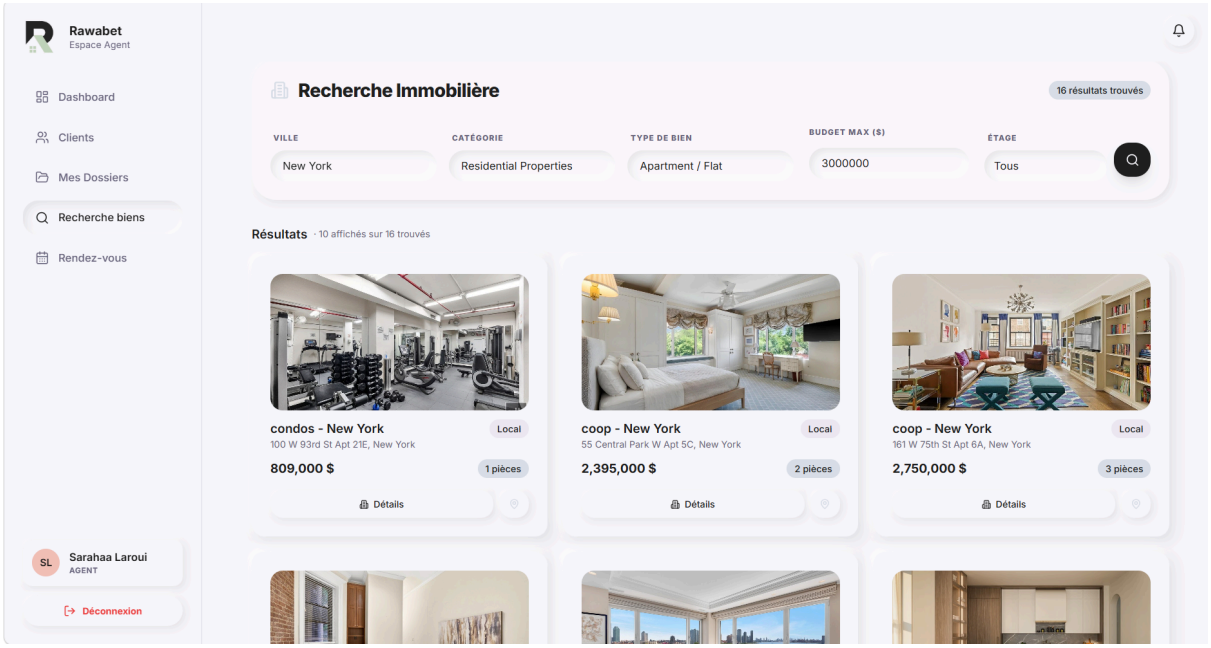
- **Recommandations immobilières générées par l'IA**



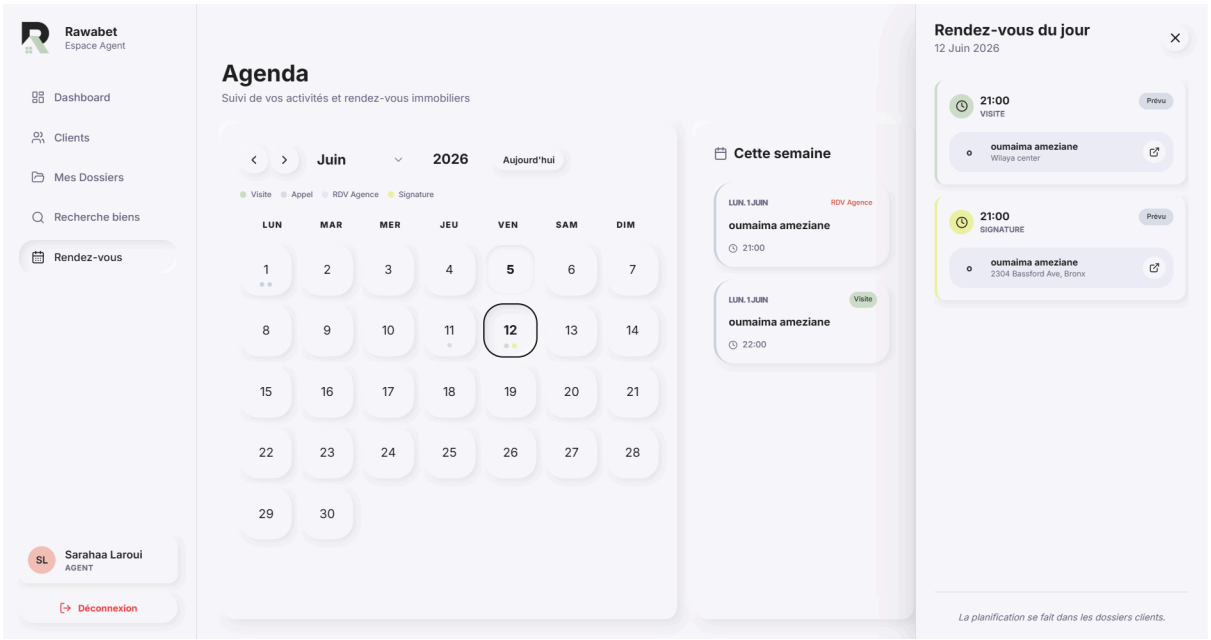
- **Interface de question sur un document (RAG)**



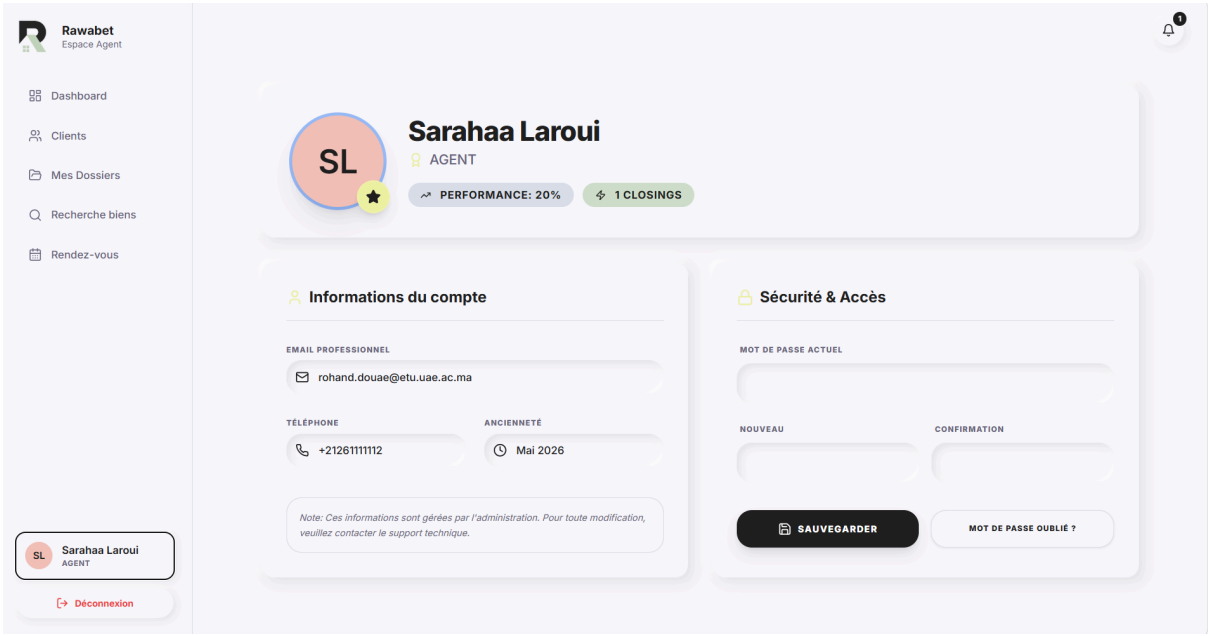
- **Recherche de propriétés avec filtres**



- **Agenda**

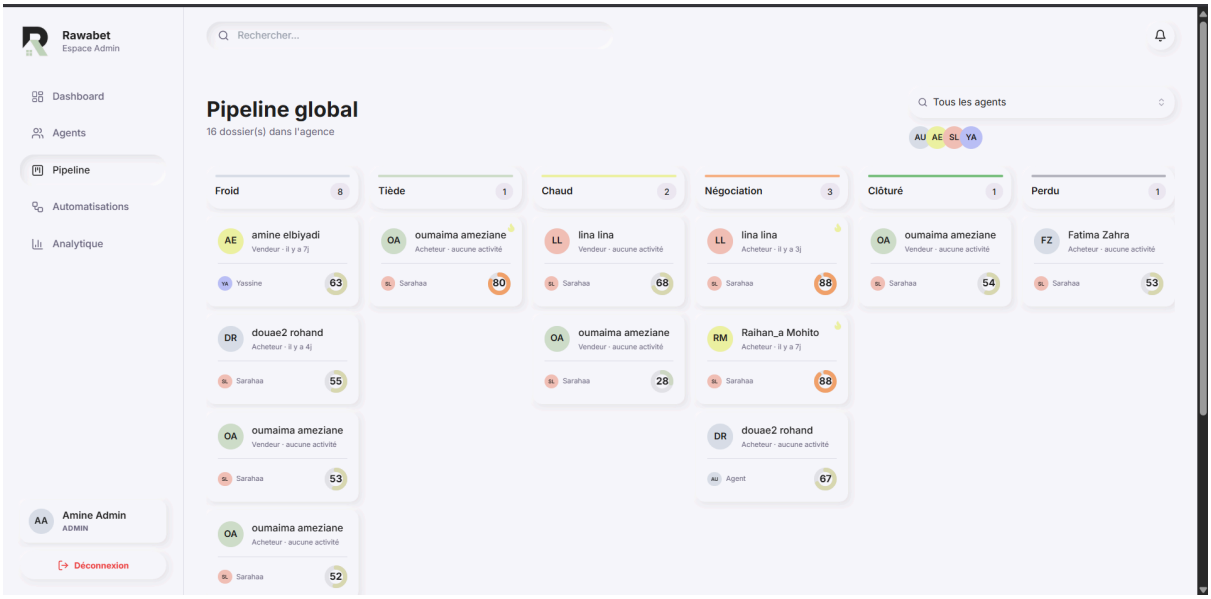


- **Espace profil et sécurité**

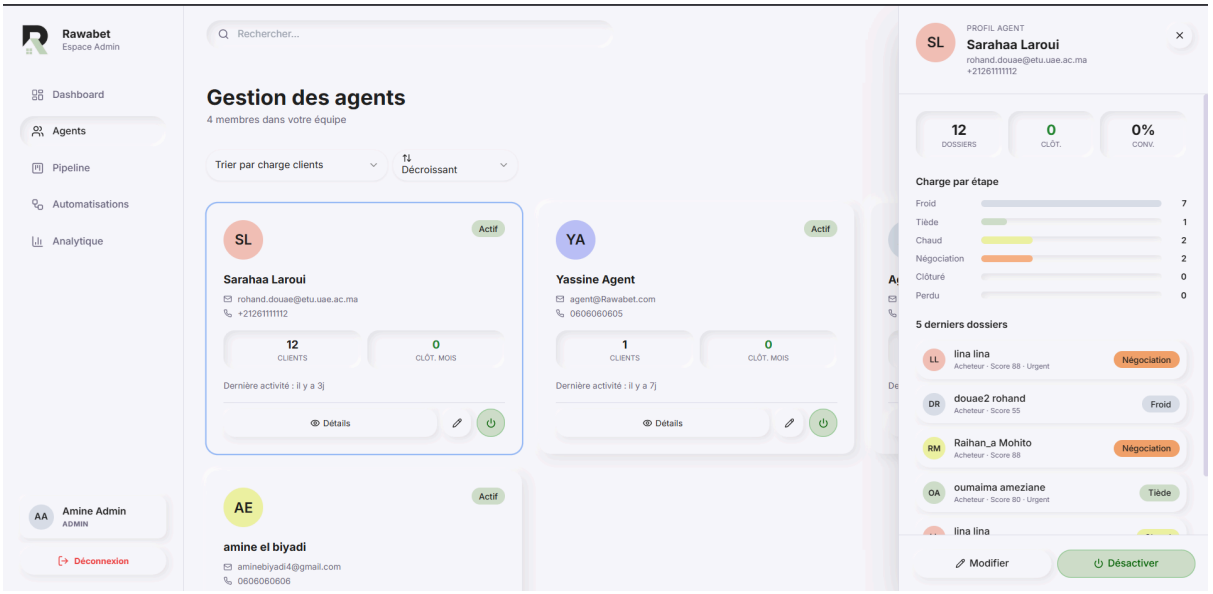


5.7.4 Espace Admin

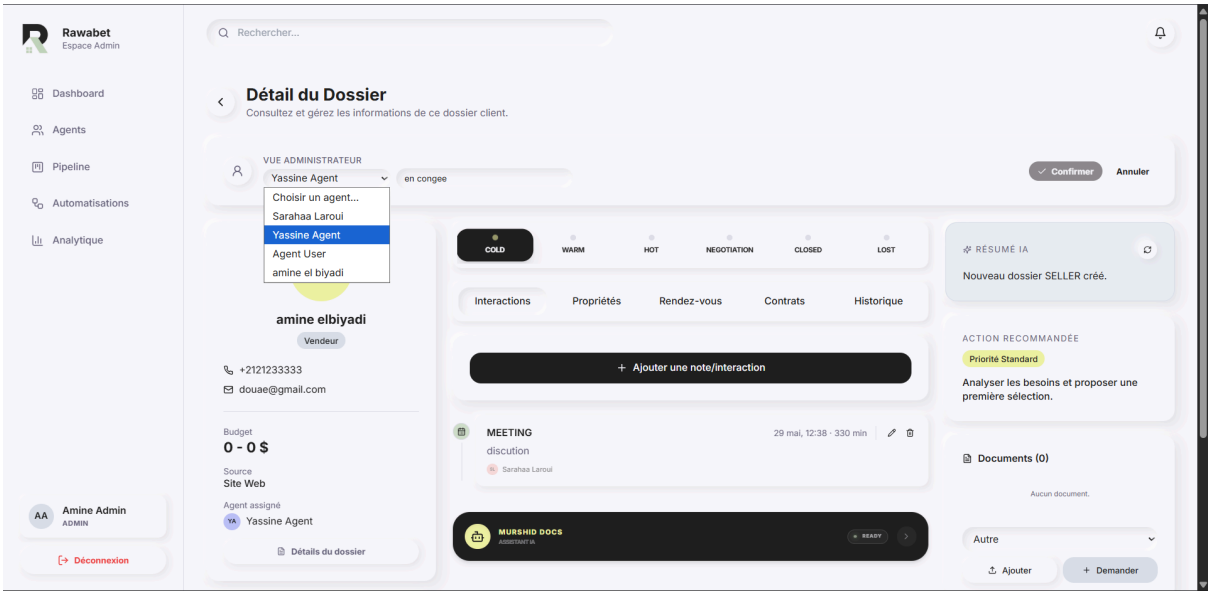
- **Dashboard admin (vue globale du pipeline tous agents)**



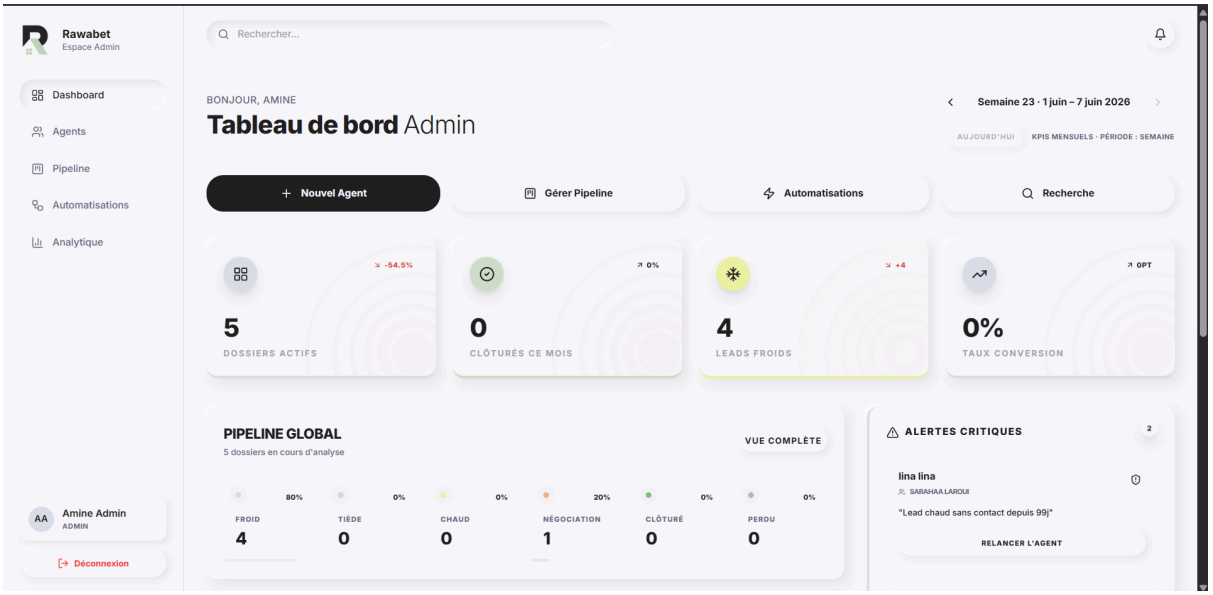
- **Liste des agents avec charge de travail**



- **Réaffectation de clients entre agents**

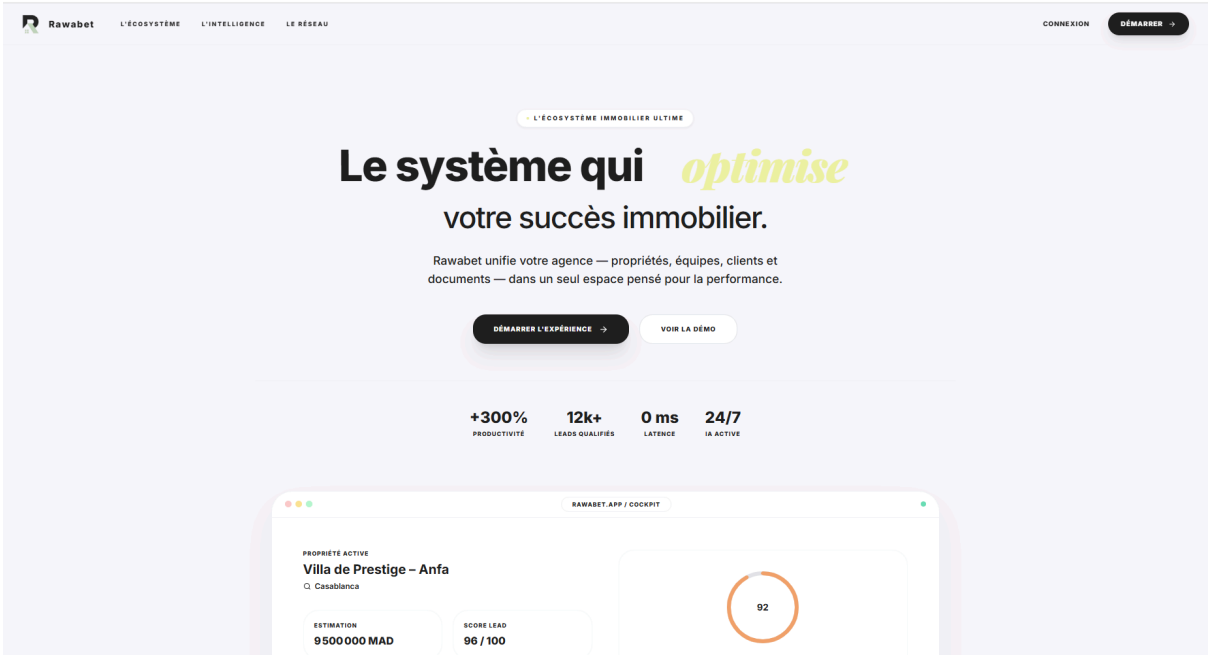


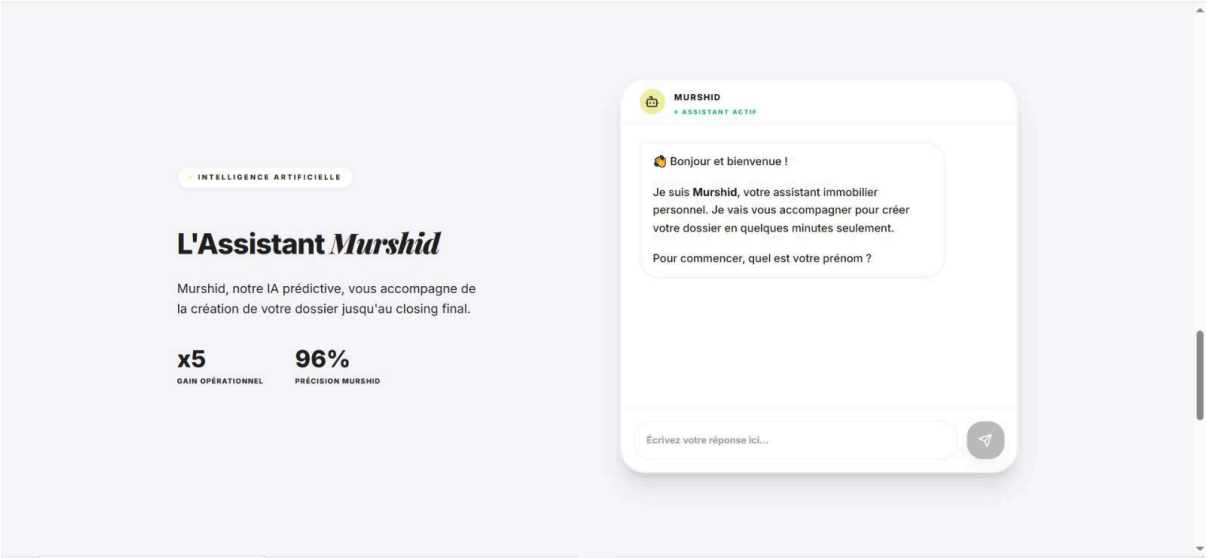
- **Tableau de bord analytique (taux de conversion, entonnoir pipeline)**



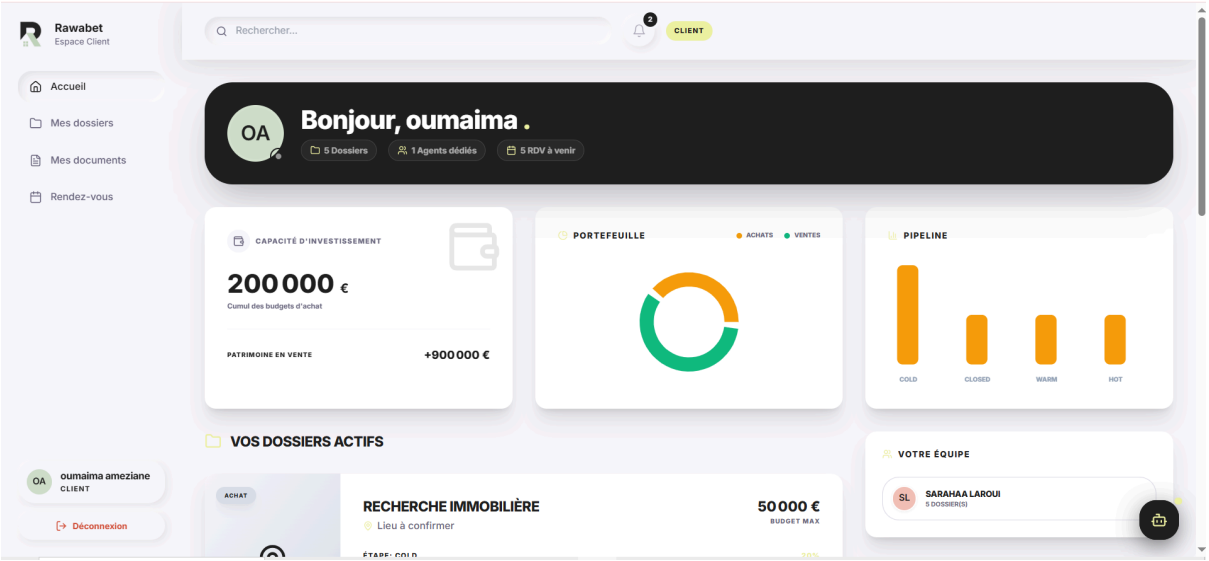
5.7.5 Portail Client

- **Landing page publique**

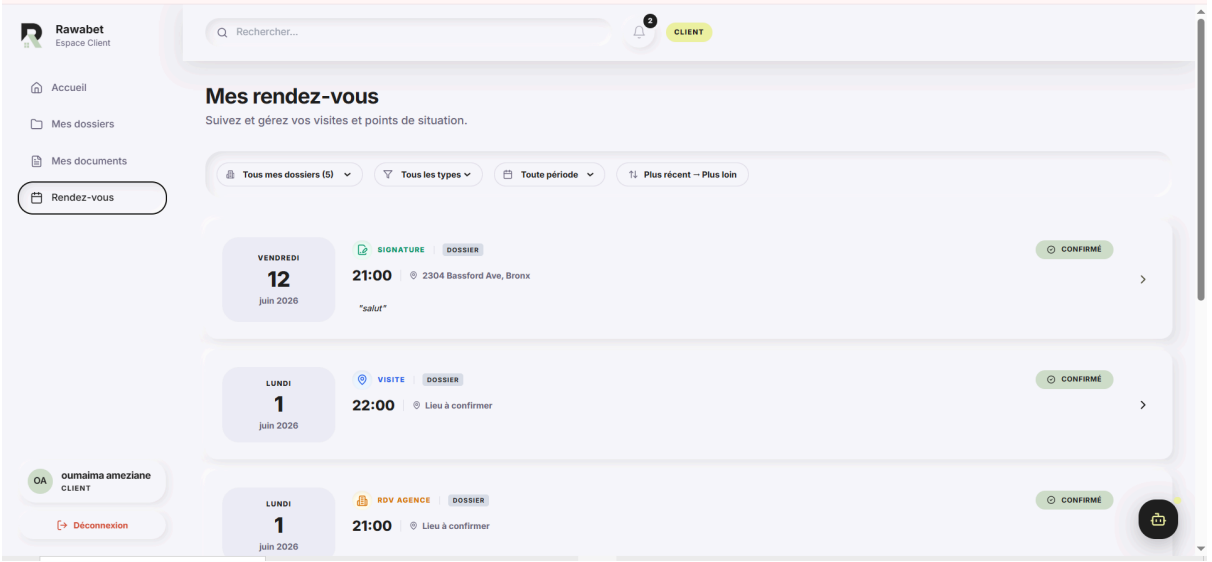




- **Tableau de bord client (indicateurs financiers, graphiques, widgets)**



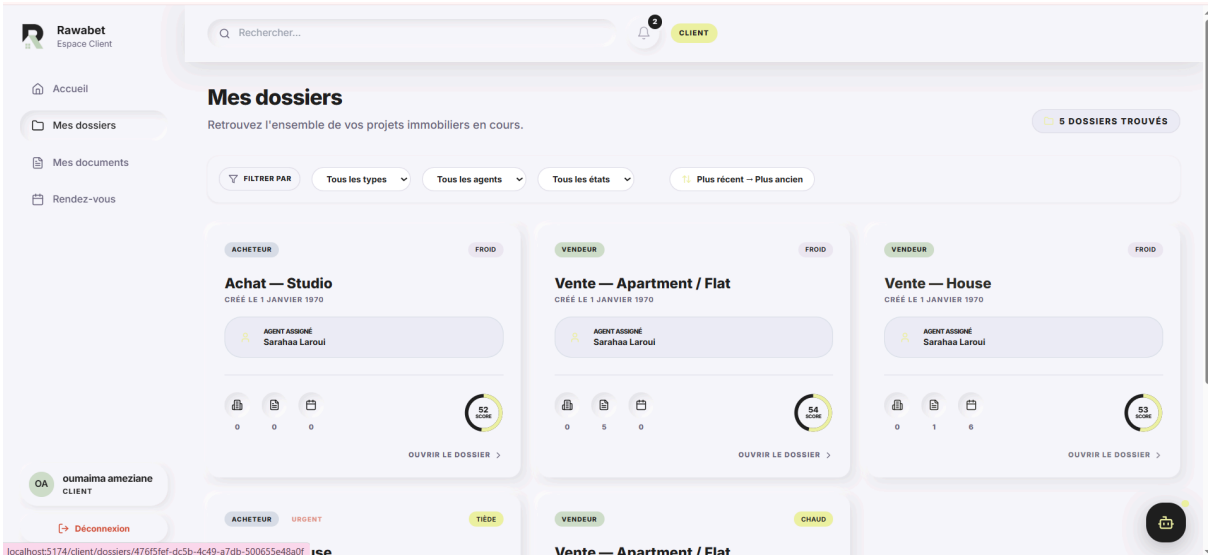
- **Liste des RDV**



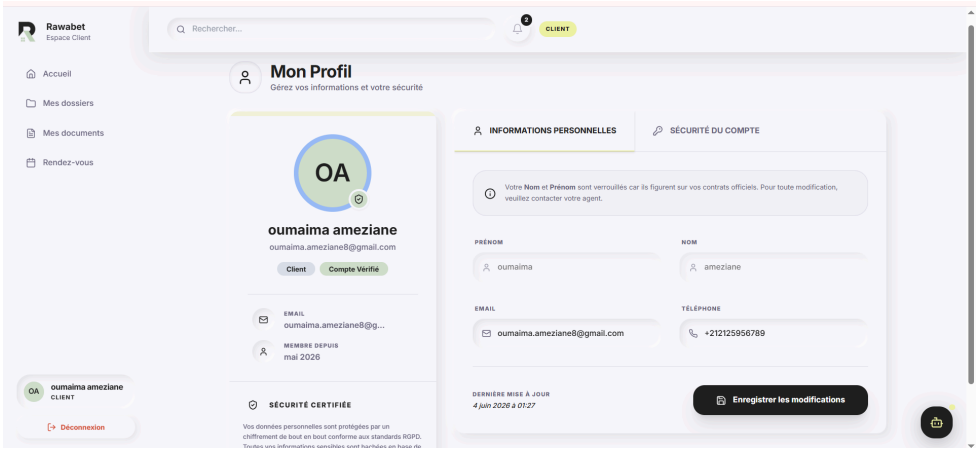
- **Liste des dossiers avec cartes et score IA circulaire**



- **Assistant MURSHID widget flottant**



- **Espace profil et sécurité**



Chapitre 6 - Sécurité

6.1 Authentification

La sécurité de Rawabet repose sur un mécanisme d'authentification stateless basé sur les JSON Web Tokens (JWT), implémenté via Spring Security avec la politique SessionCreationPolicy. STATELESS - aucune session n'est stockée côté serveur.

6.1.1 Double jeton : Access Token & Refresh Token

Le système repose sur deux jetons complémentaires :

- **Access Token :** JWT signé avec une clé secrète HMAC-512, d'une durée de vie de 24 heures. Il contient en payload l'email de l'utilisateur et son rôle (ROLE_ADMIN, ROLE_AGENT, ROLE_CLIENT).
- **Refresh Token :** Jeton opaque généré aléatoirement, stocké en base de données sous forme de hachage SHA-256 via la classe TokenHasher. Il permet de renouveler l'Access Token sans forcer une nouvelle connexion de l'utilisateur.

6.1.2 Stockage hybride des jetons

Les jetons sont transmis au frontend via des cookies HTTP-Only & Secure, ce qui prévient les attaques XSS (Cross-Site Scripting) en rendant les cookies inaccessibles depuis JavaScript. Pour assurer la compatibilité avec les clients mobiles et les APIs externes, les jetons peuvent également être transmis via l'en-tête HTTP Authorization: Bearer.

6.1.3 Authentification sociale (Google OAuth2)

La plateforme supporte la connexion via Google. Le frontend envoie un idToken Google qui est validé côté serveur via la bibliothèque officielle GoogleIdTokenVerifier. Le système vérifie l'audience et l'émetteur du token avant de lier le googleId au compte interne correspondant.

6.1.4 Hachage des mots de passe

Les mots de passe sont hachés avec **BCryptPasswordEncoder** (facteur de force 10), garantissant une protection contre les attaques par force brute et par tables arc-en-ciel (rainbow tables). Aucun mot de passe n'est jamais stocké en clair.

6.2 Autorisation

Le contrôle d'accès est implémenté selon le modèle RBAC (Role-Based Access Control) avec trois rôles métier :

Rôle	**Périmètre d'accès**
ADMIN	Accès total, gestion des agents, analytique globale

AGENT	Gestion CRM : leads, dossiers, documents, propriétés
CLIENT	Accès restreint à son propre dossier et contrats

La configuration est centralisée dans **SecurityConfig.java** via **authorizeHttpRequests**, avec les règles suivantes :

- **/api/admin/**** → hasRole('ADMIN')
- **/api/crm/**** → hasAnyRole('AGENT', 'ADMIN')
- **/api/client/**** → hasAnyRole('CLIENT', 'AGENT', 'ADMIN')

À chaque requête entrante, le filtre **JwtAuthenticationFilter.java** décode le token JWT, vérifie l'existence de l'utilisateur en base de données (et qu'il n'est pas soft-deleted) , puis peuple le SecurityContext de Spring avec l'identité et le rôle de l'utilisateur.

6.3 Protection des données et défense contre les attaques

6.3.1 Protection CSRF

Une protection CSRF (Cross-Site Request Forgery) personnalisée est implémentée dans CsrfFilter.java selon le pattern Double Submit Cookie : le serveur génère un token CSRF transmis dans un cookie, et le frontend doit le renvoyer dans l'en-tête X-CSRF-Token pour toutes les requêtes mutantes (POST, PUT, DELETE).

6.3.2 Configuration CORS

La configuration CORS est définie strictement dans **CorsConfig.java.** Seules les origines autorisées (localhost:5174, localhost:5176) peuvent envoyer des requêtes au backend. L'option allowCredentials(true) est activée pour permettre l'échange sécurisé des cookies entre le frontend et le backend.

6.3.3 Validation des entrées et gestion des exceptions

Toutes les données entrantes sont validées via les annotations JSR-303 (@Valid, @NotBlank, @Email) au niveau des contrôleurs, ce qui rejette toute tentative d'injection de données malveillantes.

Les erreurs sont interceptées par un GlobalExceptionHandler.java qui transforme les exceptions Java brutes en réponses ApiErrorResponse standardisées, masquant les stacktraces et les détails internes de la base de données.

6.4 Sécurité des APIs externes

6.4.1 Gestion des secrets

Les clés d'accès aux services tiers (JWT_SECRET, RAPIDAPI_KEY, clés OpenAI, SendGrid) sont injectées via des variables d'environnement définies dans application.yml

sous la forme ${NOM_VARIABLE}. Les fichiers .env sont utilisés en développement local et ne sont jamais committés sur le dépôt Git.

6.4.2 APIs tierces

- **RapidAPI (listings immobiliers) :** toutes les requêtes sortantes se font en HTTPS avec la clé API transmise exclusivement dans les en-têtes HTTP.
- **Cloudinary (stockage des fichiers) :** les uploads d'images sont signés côté serveur, garantissant que seuls les fichiers autorisés par la plateforme sont acceptés.
- **NVIDIA NIM (moteur IA / RAG) :** la communication avec le service d'inférence IA se fait via une clé API sécurisée pour le traitement du pipeline RAG.

6.4.3 Webhooks n8n

Les endpoints webhook exposés à n8n sont exemptés de CSRF de manière contrôlée, permettant à n8n de déclencher des actions dans la plateforme sans compromettre la sécurité globale.

Chapitre 7 - Répartition des tâches

Dans le cadre de ce projet, le travail a été réparti entre les quatre membres de l'équipe en trois phases successives : une phase de conception et modélisation, une phase de développement CRUD et logique métier, et une phase d'intégration de l'intelligence artificielle et de l'automatisation. Chaque membre s'est vu attribuer un périmètre technique distinct, couvrant à la fois le backend et le frontend, afin de favoriser la spécialisation tout en maintenant une cohérence globale du projet.

Phase 1 - Conception et modélisation

Livrable	Responsable
Diagrammes de cas d'utilisation	Oumaima Ameziane
Diagramme de classes	Douae Rohand & Raihana Mohito
Modèle Entité-Relation	Amine EL BIYADI

Phase 2 - CRUD & Logique Métier

Membre	Backend	Frontend
Amine EL BIYADI	• Module Auth : JWT, Spring Security, 3 rôles, gestion de mot de passe.	• Landing page d’agent et admin.

	● Toutes les fonctionnalités de l'espace admin: ○ Gestion des comptes agents. ○ Réaffectation des clients Supervision de la charge de travail agents. ○ Vue globale du pipeline Admin. ○ Configuration des règles métier (seuil jours inactivité, agent de secours, flag activation workflows). ○ Journal des automatisations.	● Page de connexion. ● L'espace Admin.
Douae Rohand	● Structure du projet. ● Création du profil. ● Création du dossier client complet. ● Mise à jour manuelle de l'étape du pipeline. ● Journalisation des interactions. ● Planification et gestion des rendez-vous. ● Historique chronologique des interactions et rendez-vous.	● Dashboard agent. ● Page "Mes Clients" et formulaire de création de compte client. ● Page "Dossier Clients" et formulaire du dossier. ● Formulaire journalisation interaction. ● Formulaire planification rendez-vous. ● Page de "Rendez-vous". ● profile agent
Raihana Mohito	● Recherche propriétés via RapidAPI. ● Liaison propriété au dossier. ● Création et suivi de contrats. ● Upload fichiers et métadonnées.	● Recherche et prévisualisation de propriétés. ● Liaison propriété en un clic ● Formulaire contrat et suivi statut. ● Gestion documents côté agent.
Oumaima Ameziane	● Toutes les fonctionnalités de l'espace client: ○ Consultation de ces dossiers. ○ Consultation de ces rendez-vous. ○ Upload documents côté client. ○ Download contrats et documents. ○ Notifications de confirmation de réception.	● Landing page portail client. ● Dashboard Client . ● Gestion des Rendez-vous ● Gestion des dossiers acheteurs et vendeurs ● Gestion des informations personnelles

Phase 3 - IA, Automatisation & Intégrations

Membre	Domaine	Contributions
Douae Rohand	IA - Scoring & Recommandation	● Scoring des leads (0–100) via NVIDIA NIM. ● Résumé IA des interactions. ● Suggestion automatique d'étape pipeline. ● Recommandation immobilière IA avec justification. ● Affichage score, résumé, suggestion et recommandations dans le frontend.

Oumaima Ameziane	IA - RAG & Chatbot	● Pipeline RAG complet (extraction, embeddings, pgvector). ● Endpoint questions en langage naturel sur documents (côté agent). ● Chatbot assistant client avec RAG ● Chatbot assistant pour crée un compte en attente du client et lui affecté un agent ● Interface Q&A documentaire et chatbot dans les deux frontends.
Raihana Mohito	Automatisation - n8n	● Webhooks exposés à n8n. ● 8 workflows n8n : onboarding client, Google Calendar, rappels, lead froid, suivi contrat, clôture dossier, rapport hebdomadaire, onboarding agent.
Amine EL BIYADI	Notification & Intégrations	● Module Notification complet. ● Emails transactionnels SendGrid. ● Brouillon email IA. ● Notifications in-app. ● Analyse des risques contractuels. ● Génération rapport Markdown → PDF. ● Frontend : centre de notifications, brouillon email, affichage analyse risques.

Conclusion générale

Ce projet nous a permis de concevoir et développer **Rawabet**, une plateforme CRM alimentée par intelligence artificielle, pensée spécifiquement pour répondre aux besoins réels des agences immobilières. Contrairement aux solutions génériques du marché, Rawabet intègre nativement les flux de travail propres au secteur : gestion des dossiers clients, suivi du pipeline de leads, automatisation des communications et analyse intelligente des documents.

Sur le plan architectural, nous avons opté pour un **monolithe modulaire Spring Boot,** cette approche nous a permis de maintenir une séparation claire des responsabilités entre les quatre modules: Auth, CRM Core, Moteur IA et Notification. La communication inter-modules via **Spring Events** garantit un couplage faible sans infrastructure distribuée.

L'intégration de l'intelligence artificielle constitue la valeur ajoutée centrale de la plateforme. Via **Spring AI** et les modèles **NVIDIA NIM**, Rawabet offre un scoring automatique des leads, des résumés d'interactions en langage naturel, un pipeline RAG pour l'interrogation des documents clients, et une génération automatisée de rapports hebdomadaires. Ces fonctionnalités transforment concrètement le quotidien de l'agent, qui passe moins de temps sur les tâches administratives et plus de temps sur la relation client.

L'automatisation des processus métier via **n8n** couvre les étapes critiques du cycle de vie d'un dossier : onboarding des nouveaux clients, gestion des rendez-vous, détection des leads froids, suivi des contrats et livraison des rapports - sans intervention manuelle.

Sur le plan personnel, ce projet nous a permis d'acquérir des compétences concrètes et directement valorisable: ingénierie IA appliquée (LLM, RAG, embeddings), conception d'architecture logicielle, intégration d'APIs tierces, automatisation de processus métier et développement full-stack. Chaque membre de l'équipe a contribué à un domaine technique distinct, favorisant à la fois la spécialisation et la collaboration.

Rawabet démontre qu'il est possible, dans un contexte académique et avec une équipe réduite, de livrer une plateforme métier complète, cohérente et techniquement ambitieuse - à condition de faire les bons choix d'architecture dès le départ et de maintenir une discipline rigoureuse dans la répartition des responsabilités.